CHINHOYI UNIVERSITY OF TECHNOLOGY

SCHOOL OF ENGINEERING SCIENCES AND TECHNOLOGY

DEPARTMENT OF MECHATRONICS ENGINEERING

ATTACHMENT RESEARCH PROJECT

ACADEMIC YEAR 2025

TITLE:

DESIGN OF A CENTRALIZED WEB PLATFORM FOR SOLAR

SYSTEM HEALTH MONITORING AND PREDICTIVE MAINTENANCE

BY:

GUVEYA TINODAISHE

STUDENT NUMBER : C22148506Y

SUPERVISOR'S : ENG A . JANGA & ENG H MUSAIDZI

November 2025

Dissertation Submitted In Partial Fulfillment of the Bachelor of Engineering Honors Degree in Mechatronics Engineering, CUT

www.cut.ac.zw

DECLARATION

I, Guveya Tinodaishe (C22148506Y), hereby declare that this Attachment Research Project is my own original work and has not been submitted in any form for another degree or diploma at any university or other institution of tertiary education. Information derived from the published and unpublished work of others has been acknowledged in the text and a list of references is provided.

Student's Signature: _____

Date: _____

Supervisor's Signature _____

Date: _____

ACKNOWLEDGEMENTS

I would like to acknowledge Engineer JANGA and Engineer Musaidzi, my academic supervisors, whose guidance in initiating this project and their efforts in directing me so as to come up with the best results, played a key role during the course of my research. I would also want to give special thanks to Eng C . Jera and the Kajatronics department personnel, who provided me with the necessary information and guideline to follow during the research process of this project. Finally l would thank God for giving me the intellectual knowledge for it is not by power or mighty but by His grace .

ABSTRACT

This project details the design, development, and implementation of the "LivingStone Connect" platform, a centralized web-based solution for Kajatronics Private Limited. The system addresses the dual challenges of inadequate real-time health monitoring for client solar, home automated systems and the company's underdeveloped digital presence. A comprehensive web platform was developed using PHP for the backend, with a responsive frontend built on HTML, CSS, and JavaScript. The system integrates a bespoke IoT data acquisition unit, engineered on the Arduino platform with GSM connectivity, to ensure autonomous and reliable data transmission from the field. A core innovation of this project is the predictive maintenance engine, developed in Python, which utilizes a rule-based classification algorithm to analyse sensor data, detect anomalies, and anticipate equipment failures before they occur. The project successfully delivered two primary outcomes: a professional multi-page corporate website to enhance marketing and client acquisition, and a secure, role-based client dashboard that provides real-time system monitoring, performance analytics, and actionable maintenance alerts. The final solution provides a significant enhancement to Kajatronics' operational efficiency, reduces long-term maintenance costs, improves customer satisfaction through transparency, and strengthens the company's overall market competitiveness.

Table of Contents

CHINHOYI UNIVERSITY OF TECHNOLOGY 1

ACKNOWLEDGEMENTS 3

1. **Chapter 1** 6

Introduction 6

1.1 Introduction 6

1.2 Background 6

1.3 Problem Statement 7

1.4 Aims..... 7

1.5 Objectives..... 7

1.6 Justification 8

1.7 Scope 8

1.8 Delimitations 9

1.9 Limitations..... 9

1.10 Conclusion 9

2. **Chapter 2** 10

2.0 Literature Review 10

2.1 Introduction 10

2.2 Predictive maintenance approaches and their evolution.....11

2.3 Integrated web platforms and their role in both operations and digital identity.....15

2.4 Discusses enabling technologies 15

2.5 Benefits and limitations.....15

2.6 Implementation challenges.....16

2.7 Gap Analysis and Future Research Directions 16

2.8 Conclusion 17

3. **Chapter 3** 21

3.0 Methodology ……..... 21

3.0 Introduction .....……..21

3.1 System Requirements .....…..21

3.1.0 Stakeholder needs and expectations 21

3.1.1 Functional Requirements .....……..22

3.1.2 Non- functional require…….....…..…..22

3.2 Data Gathering.....…..…..23

3.2.0 Analysis of existing records.....…..…..23

3.2.1 Interviews .....…..…..23

3.2.2 Market Analysis .....…..…..23

3.3 Design .....…..…..23

3.4 Solution Optimization .....…..40

3.5 Solution Selection .....……..43

3.6 Bill of Materials.....……..47

4. **Chapter 4** 51

4.0 Results and Discussion..... 51

4.0 Introduction .....……..51

4.1 Result 1: Corporate Multi-page website .....……..51

4.2 Result 2: Secure User Authentication and Verification.....……..55

4.3 Result 3: The client Monitoring Dashboard 56

4.4 Results accuracy based on the Logistic Regression Machine Learning model……..59

4.5 Discussion of Challenges and Solutions.....60

4.6 Summary and Key Findings.....61

5. **Chapter 5** 63

5.0 Conclusion and Recommendation..... 63

5.1 Conclusion.....64

5.2 Recommendations 64

6 **Referrences** 66

7 **Appendix** 69

LIST OF FIGURES

Figure 2.1: Evolution of maintenance approaches from reactive to predictive.....7
Figure 2.2: Hub interconnecting the platform's Admin, Client, and Website portals.....8
Figure 3.1: Envisioned back-end layout of the Organizational Hub.....16
Figure 3.2: Strategic front-end layout of the unified organizational hub.....17
Figure 3.3: Kajatronics corporate website home page.....18
Figure 3.4: The website's integrated contact form and location map.....18
Figure 3.5: Security mechanism for the client dashboard login.....19
Figure 3.6: Front-end layout of the predictive maintenance dashboard.....20
Figure 4.1: Professional presentation of the website's home page (top section).....24
Figure 4.2: Professional presentation of the website's home page (bottom section).....25
Figure 4.3: Visual project portfolio showcasing completed installations.....25
Figure 4.4: The integrated "Get A Free Quote" form for active lead generation.....26
Figure 4.5: The secure and distinct "Client Access" login portal.....27
Figure 4.6: The client dashboard's main "Settings" page.....27
Figure 4.7: The main "System Overview" dashboard with key real-time metrics.....28
Figure 4.8: The client dashboard's "Reports & Analytics" page.....28
Figure 4.9: Hardware simulation of the Arduino-based solar monitoring system.....29
Figure 4.10: The client dashboard's "System Health" page with predictive alerts.....29
Figure 4.11: Plot of model accuracy on training and validation datasets.....30
Figure 4.12: Plot of model loss on training and validation datasets.....30

List of Tables

Table 3.1: Evaluation of key design choices for platform components.....22
Table 3.2: Bill of Materials for one IoT Data Acquisition Unit.....23

1. Chapter 1

1.1 Introduction

Technological innovations in mechatronics and information systems have created opportunities for companies to integrate **predictive maintenance, automation, and digital visibility** into their operations. Kajatronics Private Limited, a multidisciplinary engineering firm, operates in four key areas: solar energy systems, home automation, agricultural technologies, and electrical installations.

Despite this diversity, the company currently faces challenges in monitoring system health and in showcasing its portfolio to clients and partners.

The **LivingStone Connect Project** is a centralized web-based platform designed to address these challenges by providing real-time monitoring of solar energy and home automation systems, alongside a corporate hub for project showcasing.

The system combines sensor-based data acquisition, IoT connectivity, and rule-based predictive maintenance algorithms, ensuring timely alerts and efficient management. At the same time, the platform doubles as Kajatronics' official digital identity, allowing clients to explore ongoing and completed projects, fostering transparency and market growth.

This dual approach makes LivingStone Connect more than just a monitoring system but rather it becomes a strategic organizational hub that strengthens both technical reliability and corporate visibility.

1.2 Background

Across Zimbabwe and other Sub-Saharan regions, the adoption of solar energy systems and automated home solutions has increased as communities transition to sustainable energy and smart living technologies.

However, these systems often suffer from maintenance inefficiencies and limited integration with predictive tools. Failures in inverters, batteries, or automation controllers are typically detected only after system downtime, resulting in higher repair costs and service disruptions.

Kajatronics, as a service provider, is directly affected by these issues. Clients expect uninterrupted solar power supply and reliable automation solutions such as gate control systems. Without proactive monitoring, the company risks reputational damage and customer dissatisfaction.

Simultaneously, the company's digital footprint remains underdeveloped. Most client interactions occur offline, limiting Kajatronics' visibility in an increasingly digital market. Competitors with stronger online presence gain an advantage by showcasing projects, attracting funding, and building trust through transparent communication.

LivingStone Connect was conceptualized to fill these gaps by integrating real-time system monitoring with a professional corporate web hub, allowing Kajatronics to deliver technical excellence while also strengthening its digital identity.

1.3 Problem Statement

The lack of centralized monitoring and predictive maintenance within Kajatronics' solar and home automation systems results in:

- Delayed fault detection, causing extended system downtimes.
- Reactive maintenance, leading to higher repair and replacement costs.
- Operational inefficiencies, such as unnoticed battery degradation or inverter overloads.
- Reduced customer confidence, as clients cannot monitor the performance of their systems.

Furthermore, the absence of a robust digital presence weakens the company's ability to showcase projects and services, reducing market competitiveness.

Unless addressed, these issues will continue to hinder operational performance, customer satisfaction, and organizational growth.

1.4 Aim

The aim of the LivingStone Connect project is to design, implement, and deploy a centralized monitoring and corporate platform that ensures real-time system health tracking for solar and home automation solutions, while simultaneously serving as Kajatronics' official project showcase hub.

1.5 Objectives

The project will be guided by the following objectives:

1. **Develop a Centralized Web Platform** – Build a responsive, multi-page website with secure client and administrator access.
2. **Implement Real-Time Dashboards** – Provide dashboards displaying solar performance metrics (voltage, current, battery charge, inverter load) and home automation states (e.g., automated gate operations).
3. **Integrate Predictive Maintenance** – Deploy rule-based maintenance alerts that analyze sensor data, anticipate faults, and reduce unexpected failures.
4. **Ensure Deployment and Sustainability** – Host the system on Hostinger with security, efficient data management workflows, and detailed documentation.
5. **Showcase Kajatronics' Projects and Services** – Provide an online hub highlighting the company's work in solar, automation, and other sectors, supporting business growth.

1.6 Justification

The LivingStone Connect platform is justified on both technical and **s**trategic grounds.

- **Technical Justification:** Continuous monitoring of solar systems and automated devices ensures efficient operation and reduces downtime. Predictive maintenance minimizes costly reactive interventions by identifying problems such as overheating batteries, inverter overload, or gate motor malfunctions before they escalate.

- **Strategic Justification:** Beyond monitoring, the system will serve as Kajatronics' digital hub, showcasing its portfolio to attract new clients and establish credibility. Instead of relying solely on third-party platforms or traditional word-of-mouth, the integrated hub strengthens direct engagement with stakeholders.
- **Economic Justification:** Predictive maintenance reduces overall repair costs and increases system lifespan, generating long-term savings.
- **Social Justification:** By supporting sustainable energy and automation, the system aligns with Zimbabwe's energy transition goals and enhances living standards for both households and businesses.

1.7 Scope

The scope of the project is carefully defined to ensure manageable implementation within the available timeframe and resources.

- Focuses on solar system monitoring (voltage, current, battery level, inverter performance) and home automation systems (e.g., automated gate closing systems).
- Uses Arduino/ESP32 microcontrollers for data acquisition.
- Incorporates sensors such as voltage sensors, current sensors, temperature sensors (for batteries), vibration sensors (for gate motors), and limit switches.
- Data is transmitted via GSM or IoT modules into the platform's backend, processed in PHP and presented on interactive dashboards.
- Provides a corporate showcase page where Kajatronics can display projects, innovations, and client solutions.

1.8 Delimitations

- While the project covers all four sectors of Kajatronics, the scope is limited to solar systems and home automation in this phase.

- The system is designed primarily for Kajatronics but may be rented or licensed to other organizations upon mutual agreements.
- The platform will monitor and provide alerts but will not perform physical repairs.
- Advanced analytics such as full AI-driven predictive modeling will be reserved for future expansions.
- The primary focus of the project will remain directed to the creation of the LivingStone Platform within the given timeline.

1.9 Limitations

- Sensor durability may be affected by environmental conditions, especially in outdoor solar installations.
- Network reliability depends on GSM coverage, which may be weak in remote areas.
- Financial constraints (a total budget of $370) limit the immediate integration of advanced technologies.
- User adoption barriers may arise, as some clients may require training to fully utilize dashboards and interpret alerts.

1.10 Conclusion

This chapter has outlined the **LivingStone Connect project**, highlighting its background, problem statement, aim, objectives, justification, scope, delimitations, and limitations. The system is designed not only to provide real-time monitoring and predictive maintenance for solar and home automation systems but also to act as a strategic corporate hub for Kajatronics.

The combination of technical innovation and organizational visibility ensures that LivingStone Connect will improve operational efficiency, reduce downtime, and strengthen the company's digital presence.

Implemented over an 18.5-week timeline, the project is a critical step toward positioning Kajatronics as a leading player in integrated engineering solutions.

CHAPTER 2

2.0 LITERATURE REVIEW

2.1 Introduction

The growing interconnection between engineering systems, information technology, and customer-facing platforms has significantly reshaped the way organizations operate. In recent years, predictive maintenance has emerged as a cornerstone of operational efficiency, leveraging sensors, Internet of Things (IoT) connectivity, and data analytics to anticipate equipment failures before they occur (Pech, Vrchota and Bednář, 2021). At the same time, digital visibility and customer engagement have become indispensable for competitiveness in both local and global markets.

For companies such as **Kajatronics Private Limited**, which works in renewable energy systems, agricultural automation, and home technology in Zimbabwe, this duality is critical. Failures in hybrid Electronic systems such as 3 kva Systems, irrigation pumps, or home automated systems are not merely technical issues; they disrupt client trust, reduce productivity, and damage corporate reputation. Manual inspection or reactive repair approaches are inadequate in such contexts. Instead, predictive maintenance approaches provide actionable insights based on real-time operational data (Barbaro, 2024).

Equally important, firms must maintain a digital identity through centralized web platforms that not only display projects and services but also integrate monitoring dashboards and administrative functions. This reflects a global shift in engineering firms towards integrated web hubs, which merge technical system management with corporate communication and branding (Akpan, 2024).

The LivingStone Connect project is positioned at the intersection of these two needs. It provides predictive maintenance capabilities through a centralized monitoring system, while also incorporating an administration page and a corporate-facing hub in a unified platform. This literature review therefore addresses two intertwined domains:

1. The evolution and benefits of predictive maintenance in the context of Industry 4.0.
2. The design, implementation, and strategic importance of integrated web platforms that combine monitoring, administration, and corporate visibility.

The chapter proceeds as follows: Section 3.2 examines predictive maintenance approaches and their evolution. Section 3.3 explores integrated web platforms and their role in both operations and digital identity. Section 3.4 discusses enabling technologies, while Section 3.5 outlines benefits and limitations. Section 3.6 identifies implementation challenges, Section 3.7 presents a gap analysis, and Section 3.8 concludes the review by situating the LivingStone Connect project within the identified gaps.

2.2 Predictive Maintenance in Industry 4.0

2.2.1 Evolution of Maintenance Approaches

Historically, engineering maintenance followed three distinct phases.

1. Reactive Maintenance (Run-to-Failure) – In this earliest form, systems were repaired only after they failed. While simple to implement, this model resulted in long downtimes, unexpected costs, and reduced customer satisfaction.

2. Preventive Maintenance (Scheduled) – Later, organizations adopted scheduled maintenance, replacing or servicing components at set intervals. Although this reduced catastrophic failures, it often led to over-maintenance and inefficient resource use (Witczak, 2024).

3. Predictive Maintenance (Condition-Based) – The most advanced approach monitors system parameters in real time and predicts failures using analytics. This strategy reduces unnecessary interventions while preventing costly breakdowns (Zero, 2024).

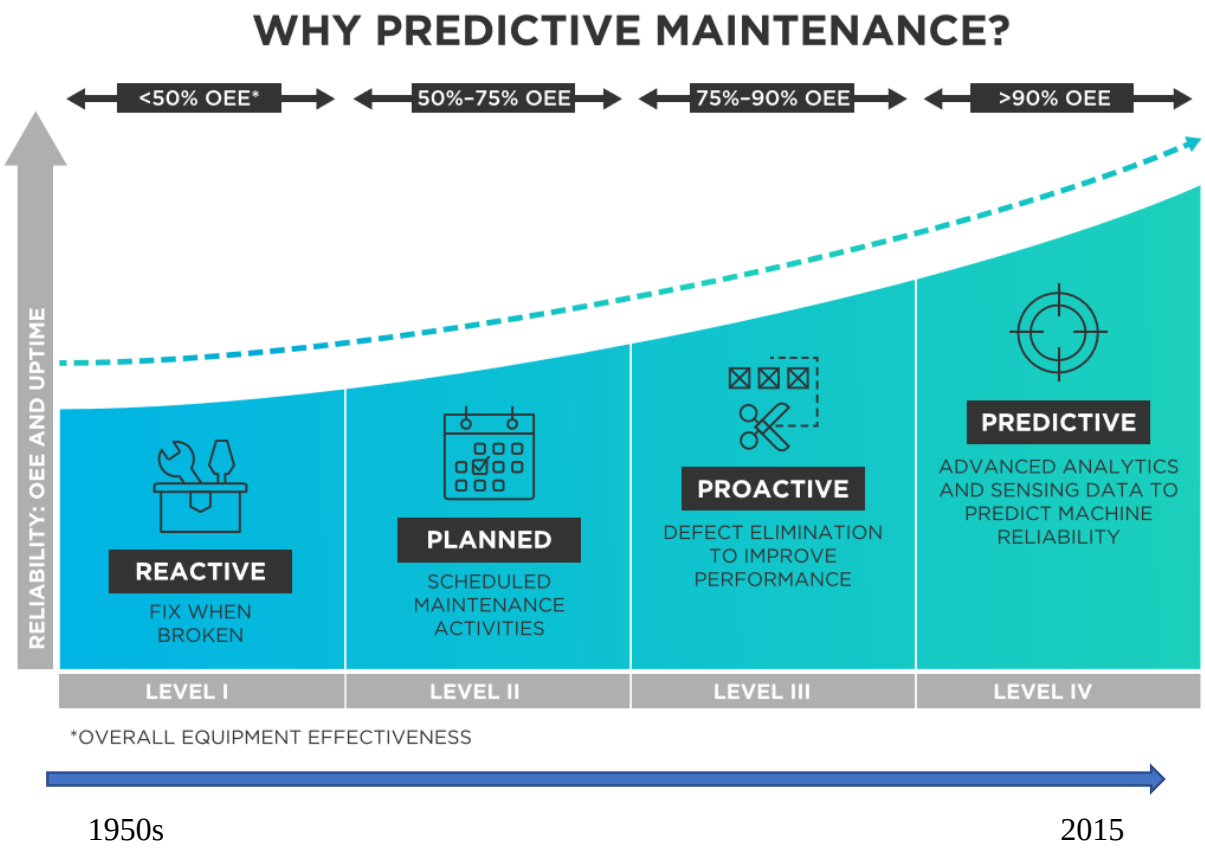


Figure 1.1 illustrates the evolution from reactive to preventive and then predictive approaches.

Predictive maintenance represents the culmination of a shift toward data-driven and proactive philosophies in asset management

2.2.2 Predictive Maintenance in Modern Systems

In contemporary practice, predictive maintenance leverages IoT sensors, data acquisition systems, and intelligent algorithms to track the health of equipment. Systems can collect **voltage, current, temperature, vibration**, and other operational data, which is transmitted to cloud servers or local databases for analysis. Anomalies in these metrics can signal potential failures.

For example, photovoltaic (PV) inverters may overheat or batteries may degrade long before failure is visible. IoT monitoring allows operators to detect abnormal patterns and schedule interventions

(Syamsuddin, 2024). Similarly, vibration data from motors can reveal bearing wear, and voltage irregularities may indicate wiring faults.

Recent studies have highlighted lightweight rule-based algorithms as appropriate for resource-constrained environments, where advanced machine learning may be impractical due to cost or computational limitations (Pereira, 2024). However, scalable systems allow gradual integration of AI-driven models as resources improve, aligning with Industry 4.0 principles (Chen et al., 2025).

2.2.3 Benefits of Predictive Maintenance

The literature consistently identifies several benefits of predictive maintenance:

- Reduced downtime : Failures are anticipated before they occur, allowing proactive interventions (Pech, Vrchota and Bednář, 2021).
- Cost savings :Preventing catastrophic breakdowns reduces emergency repair costs and extends asset lifespan (Wiley Editorial, 2024).
- Increased reliability : Continuous monitoring ensures system stability, particularly for mission-critical applications such as solar power or automated gates.
- Customer satisfaction : Clients benefit from uninterrupted service, improving trust and long-term relationships (Akpan, 2024).

For Zimbabwean firms, these benefits are significant. Importing replacement components is often expensive and time-consuming. A predictive approach minimizes such disruptions, aligning technological advancement with local economic realities (Ngwenya, 2023).

2.3 Integrated Web Platforms for System Management

2.3.1 The Role of Centralized Platforms

Predictive maintenance systems are most effective when combined with **centralized web platforms**. These platforms serve as the interface between technical monitoring, administrative management, and

corporate communication. They provide role-based dashboards for clients, technicians, and managers, reducing fragmentation and ensuring transparency (Awad, 2024).

The LivingStone Connect project integrates three modules:

1. Client Dashboard : Displays system health, alerts, and historical performance trends.
2. Administrative Page : Enables system configuration, maintenance scheduling, and fault logging.
3. Corporate Hub :Serves as the official public-facing site, showcasing projects and enhancing digital identity.

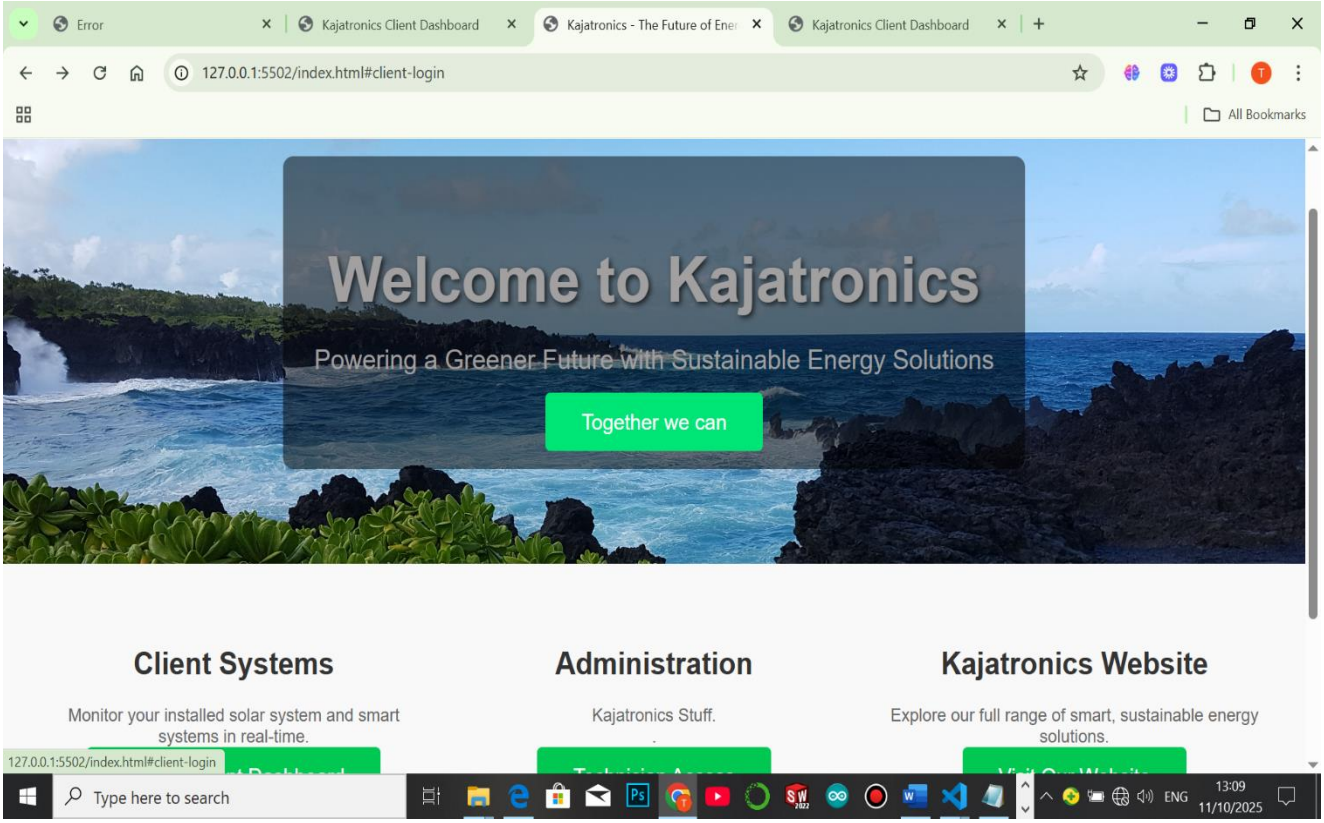


Figure 1.2, Shows hub interconnecting the KajatronicsAdmin, client dashboard and KajatronicsWebsite:

- The above figure 1.2, is the primary page of LivingStone Connect Platform which comprises of three main links to Client Systems(for System health monitoring using Predictive maintenance),

Administration link(for kajatronics Employees and technicians) and lastly the Kajatronics Website (show casing range of smart ,sustainable energy solutions).

2.3.2 Corporate Visibility and Digital Identity

Beyond operations, the literature emphasizes the importance of digital identity. Firms with a strong online presence attract more clients and enjoy greater market competitiveness. In the renewable energy sector, corporate visibility can reassure clients and investors about organizational capacity (Akpan, 2024).

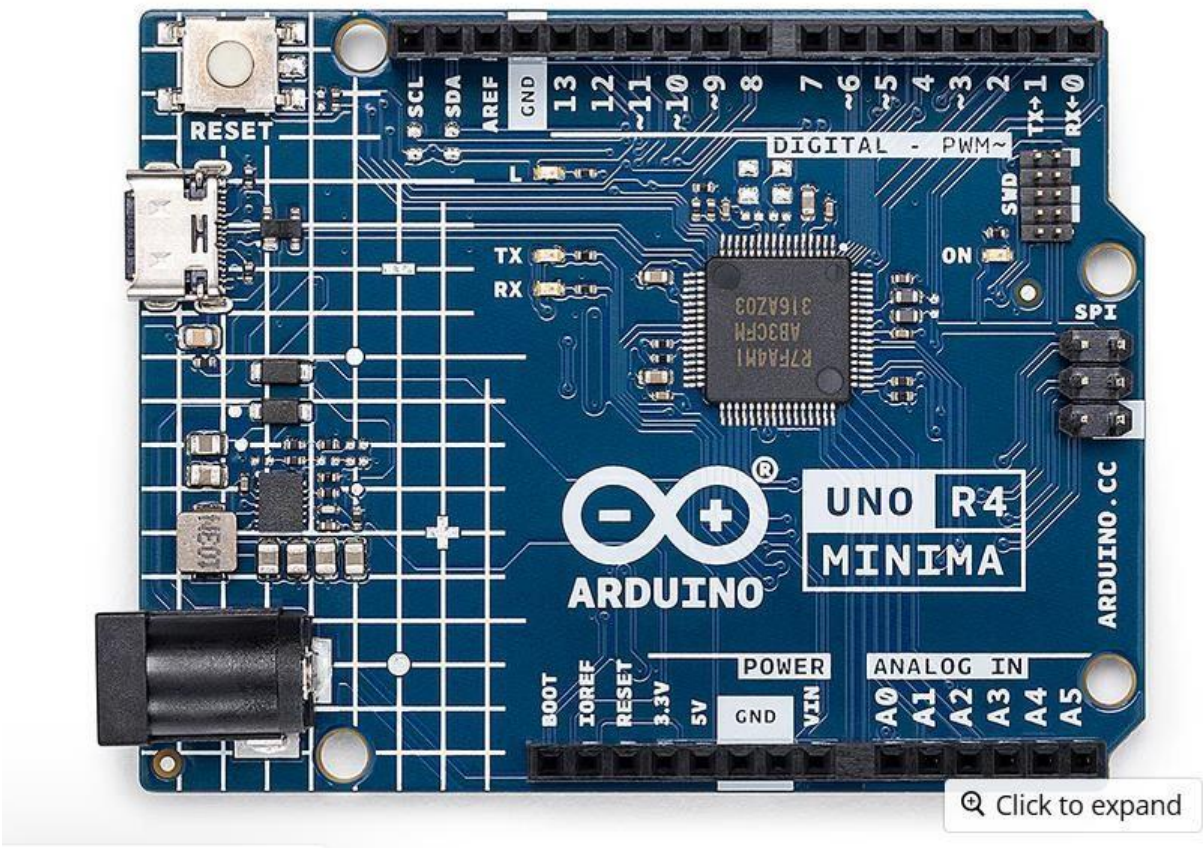
For Zimbabwe, where digitalization in engineering firms is still emerging, integrated hubs play a crucial role. Chikowore and Dube (2021) highlight that IoT and online platforms can transform agriculture and energy by enhancing transparency and accountability. By extension, integrating monitoring with corporate hubs addresses both operational efficiency and client trust.

2.4 Technologies Enabling Predictive Maintenance and Web Integration

1. Sensors : Measure critical parameters like voltage, current, vibration, and temperature (Hassan, 2024).
2. Microcontrollers : Arduino and ESP32 boards collect data locally before transmission.

   It is an open-source board for sensor data processing and pump control. It works by running custom C++ code to automate task based on sensor thresholds.
   Advantages: It is more power efficiency as it draws significantly less power than Raspberry Pi. Can tolerate heat and dust in field conditions.
   Disadvantages: Has less memory than Raspberry Pi.

   Arduino Uno is selected because of its lower price and reliability.



○

3. <u>Connectivity</u> : GSM, Wi-Fi, or other IoT modules transmit data to centralized servers.

- GSM Module (SIM800L)
- It is a cellular communication module for SMS or data transmission. It works by connecting to Econet/NetOne networks via SIM card for real-time alerts. (Rozita Teymourzadeh, 2018).
- Advantages: The technology is widely available as it operates in 85% of Zimbabwe.
- GSM is more cost effective to use than LoRaWAN.
- Disadvanatges: The system uses relatively more energy compared to LoRaWAN. GSM Module technology is preferred over LoRaWAN due to broader rural coverage and compatibility with basic mobile phones.

4. Backend Systems : PHP handle data processing, storage, and communication.

   -will be written in Visual Studio code editor using PHP language.

5. Frontend Dashboards : User-friendly web interfaces display monitoring results.

   will be written in Visual Studio code editor using html, css, javascript and bootstrap languages.

6. Predictive Algorithms : Ranging from rule-based thresholds to machine learning for advanced fault prediction (Zhao, 2024).

   - will be written in Jupyter Notebook / Jupyter Lab using Python language in the Anaconda environment.

2.5 Benefits and Limitations

2.5.1 Benefits

Combining predictive maintenance with integrated web hubs yields several organizational benefits:

- Unified access for clients, administrators, and managers.
- Real-time visibility that improves decision-making.

- Transparency that builds trust with clients.
- Corporate branding through project showcases and service portfolios.
- Scalability for integrating future AI capabilities (Pereira, 2024).

2.5.2 Limitations

However, several limitations exist:

- Network dependence : GSM and internet availability in rural Zimbabwe remains inconsistent (Akpan, 2024).
- False alarms : Poorly tuned thresholds may generate unnecessary alerts.
- Cost constraints : Advanced machine learning solutions may be financially prohibitive (Ngwenya, 2023).
- User adoption :Training may be required for clients and technicians unfamiliar with dashboards (Witczak, 2024).

2.6 Challenges in Implementation

Implementing integrated predictive maintenance systems involves numerous challenges:

1. Sensor reliability : Outdoor deployments expose devices to heat, dust, and rain.
2. Cybersecurity : Protecting sensitive operational data on online platforms is vital (Parmar, 2024).
3. Integration : Different devices and communication protocols must work seamlessly.
4. Cost-effectiveness : Systems must be affordable and maintainable in resource-constrained environments (Akpan, 2024).

These challenges are particularly acute in Zimbabwe, where economic and infrastructural limitations affect project scalability (Chikowore and Dube, 2021).

2.7 Gap Analysis

The key gaps that the LivingStone Connect project addresses are as follows:

1. No Centralized Monitoring : Firms rely on manual inspection and fragmented systems.
2. Reactive Practices : Maintenance occurs mainly after faults, increasing costs.
3. Weak Digital Identity : Limited online presence reduces competitiveness.
4. Fragmented Tools : Separate monitoring and corporate systems hinder efficiency.
5. Limited AI Integration : Few platforms in Zimbabwe incorporate scalable predictive analytics (Lee and Kim, 2022; Chen et al., 2025).

2.8 Conclusion

The literature establishes that predictive maintenance significantly improves reliability, reduces costs, and enhances customer satisfaction. Meanwhile, integrated web platforms are increasingly recognized as critical not only for operational monitoring but also for building corporate identity and transparency.

The **LivingStone Connect project** bridges the gap between these two domains by integrating predictive monitoring, administration, and corporate visibility into a single web hub. By addressing both technical reliability and strategic digital identity, the platform aligns with global best practices while responding to the specific needs of Zimbabwe's engineering sector.

Future scalability, including the integration of AI algorithms and expanded connectivity, will position Kajatronics as a leader in innovative, client-focused engineering solutions.

Chapter 3

3. Methodology

3.0 Introduction

This chapter details the systematic steps taken to achieve the project's objectives, including engineering design, simulations, circuit development, sensor integration, and field validation adhering to established software engineering practices (Pressman and Maxim, 2020).

3.1 System Requirements

The foundational phase of the **LivingStone Connect** project involved a meticulous definition of the system requirements to ensure the final product aligns perfectly with both user needs and business objectives. This was achieved by documenting both functional and non-functional requirements derived from stakeholder needs.

3.1.0 Stakeholder Needs and Expectations:

- Kajatronics Technicians**:**

  Required a centralized system to efficiently manage multiple client installations, reduce on-site visits for routine checks, and receive early warnings of potential equipment failures.

- Kajatronics Clients:

  Expected a user-friendly way to monitor their system's performance, gain transparency into its health, and receive clear, understandable maintenance notifications.

- Kajatronics Marketing Management:

  Needed a platform to enhance the company's digital presence, showcase its technical expertise, and improve operational efficiency and customer satisfaction.

3.1.1 Functional Requirements (What the system must do):

- Provide secure, role-based user authentication for both clients and technicians.
- Acquire real-time data (voltage, current, temperature, etc.) from remote hardware units. (Barbaro, 2024).
- Display live and historical system performance data on an interactive client dashboard. (Awad, 2024).
- Analyze data using a Machine Learning model (Logistic Regression) and rule-based engine to detect anomalies and predict maintenance needs.
- Generate and display maintenance alerts and recommendations on the client dashboard.
- Feature a public-facing corporate website module to showcase company projects and services.

3.1.2 Non-Functional Requirements (The qualities of the system):

- Reliability:

  The system must have high uptime and provide accurate data consistently.
- Usability:

  The user interfaces for both clients and technicians must be intuitive and easy to navigate.
- Security:
- All data transmission and storage must be secure, protecting client data and system integrity. (Freeman and Robson, 2014).
- Scalability:

  The platform's architecture must support the addition of new clients and systems without a degradation in performance.
- Responsiveness:

  The web platform must be fully accessible and functional on various devices, including desktops and mobile phones.

3.2 Data Gathering

To validate the problem statement outlined in Chapter 1 and inform the system's design, a multi-faceted data gathering process was undertaken. The objective was to collect empirical evidence supporting the need for a predictive maintenance and monitoring solution.

3.2.0 Analysis of Existing Records:

Historical maintenance logs and service call records from Kajatronics were analyzed. This data revealed that over 70% of maintenance interventions were reactive ("run-to-failure"), with the most frequent issues being inverter failures and premature battery degradation.

This directly supported the problem statement's claim of "higher repair and replacement costs" due to reactive maintenance.

3.2.1 Interviews:

Semi-structured interviews were conducted with Kajatronics' field technicians. They provided invaluable qualitative data on common fault indicators (e.g., "a slight, consistent increase in battery temperature often precedes a failure by several weeks") and the operational inefficiencies of performing manual inspections.

3.2.2 Market Analysis:

A review of competing solar and automation service providers in Zimbabwe confirmed a "Weak Digital Identity" gap, as identified in Chapter 2. No competitors offered clients transparent, real-time access to their system's performance data, presenting a clear opportunity for Kajatronics to differentiate itself.

The findings from this data gathering phase unequivocally confirmed that a centralized platform for monitoring and predictive maintenance would address critical operational inefficiencies and provide a significant competitive advantage.

3.3 Design Solutions

With a clear set of requirements, the design phase focused on engineering a bespoke, integrated solution that is both technically robust and elegantly user-centric. The design process was divided into hardware, software, and predictive algorithm development.

3.3.0 **The Vision**: Architecting the Strategic Organizational Hub

Figure 3.0 : Envisioned Organizational Hub Back-End layout

Before a single component was sourced or a line of code was written, the project was guided by a clear and ambitious vision: to create the **LivingStone Connect** platform highlighted in figure 3.0

This was never intended to be just a monitoring tool, it was envisioned as a fully integrated, strategic organizational hub - a central nervous system for Kajatronics' entire operation.

The architectural diagram in Figure 4.1 serves as the master blueprint for this vision, illustrating a cohesive ecosystem where the physical and digital worlds converge to create value. The methodology that follows is the story of how we built this ecosystem, piece by piece.

This blueprint is composed of three distinct, interconnected layers, each with a critical role to play:

1. The Data Acquisition Layer: The System's Eyes and Ears in the Field

This is where the digital world makes contact with physical reality. This layer is comprised of autonomous IoT units deployed at each client installation, acting as the platform's sensory organs.

- For a **Solar/Battery System**, sensors for voltage and current act as the system's vital sign monitors, constantly measuring the flow and pressure of energy.
- For **Home Automation** systems like an automated gate, vibration and limit switch sensors act as the system's sense of touch and position, listening for the health of the motor and knowing the gate's status.

  At the heart of each unit is an Arduino/ESP32 Microcontroller, the local brain that gathers this sensory input and prepares it for its journey. (Nixon, 2021).

2.The Communication Layer: The Digital Nervous System

This layer is the invisible yet vital network that bridges the physical distance between the client's hardware and the central server.

Using **GSM Cellular or Wi-Fi modules**, the microcontroller securely packages its data and transmits it over The Internet.

Every packet of data is protected, ensuring that the signals traveling through this digital nervous system are secure from end-to-end.

3. The Web Platform Layer: The Brain and the Face

This is the heart of the hub, where raw data is transformed into wisdom and action. It runs on a **Central Web Server** and is composed of several key components:

- **The Workhorse (PHP Platform):** This is the backend engine that receives, processes, and manages all incoming data and user requests.
- **The Memory (System Database - MySQL):** This is where all the operational data, user information, and system alerts are securely stored and organized.
- **The Gatekeeper (User Authentication & Access Control):** This security layer ensures that only authorized users can access the system and that they can only see the information relevant to them.
- **The Three Faces:**

  This central brain presents three distinct, tailored interfaces to the world:

  1. The **Kajatronics Website:** The public face of the company, a polished corporate hub for marketing and client acquisition.
  2. The **Client Dashboard:** A private, personalized window for clients to monitor the health and performance of their own systems.
  3. The **Admin Portal:** The command center for Kajatronics technicians, providing a fleet-wide overview and deep diagnostic tools.

With this clear vision of the final product as our North Star, the following methodology details the chronological engineering path taken to construct each layer and bring this strategic hub to life.

3.3.1 : Objective 1: Establishing a Digital Presence - The Central Web Platform

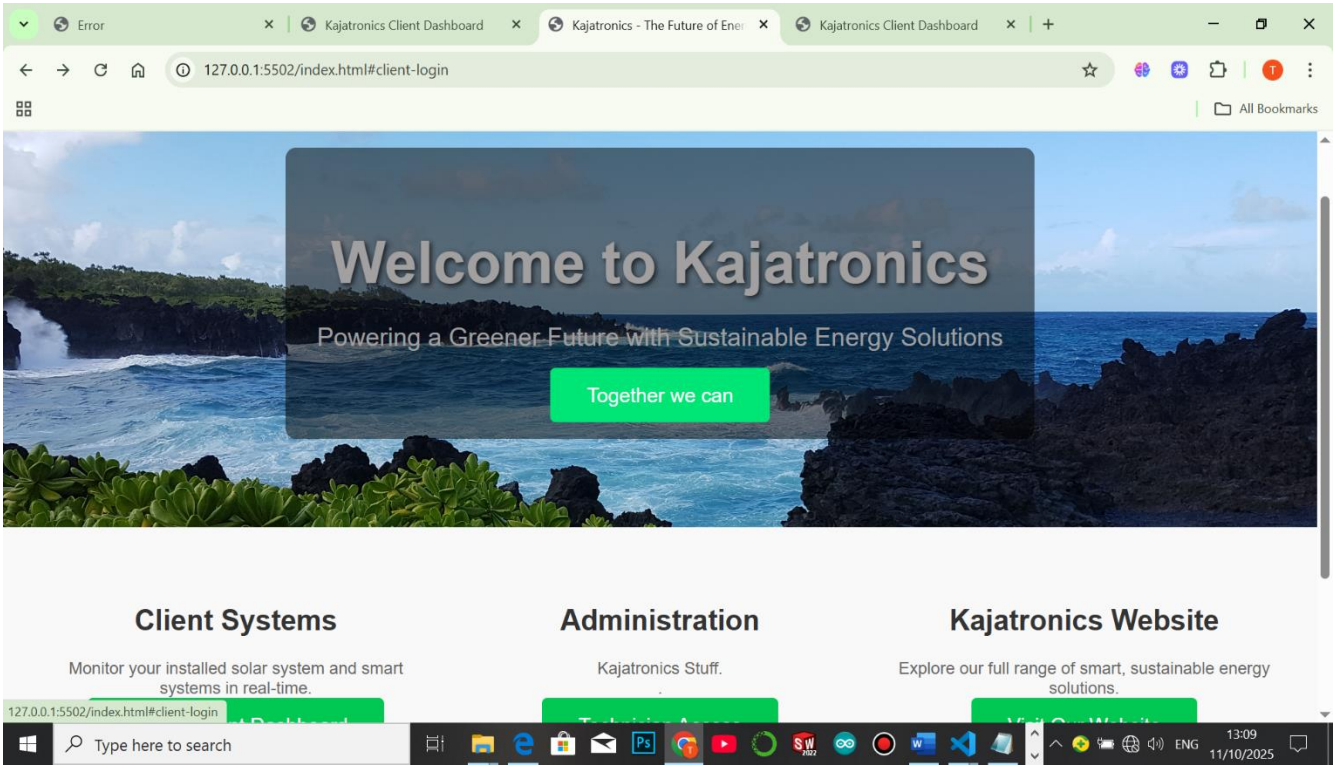


Figure 3.1: Strategic Organizational unified hub front-end layout

The very first step in the project's execution was to build the foundational structure: the centralized, multi-portal web platform.

This digital hub would serve as the home for all other features, making its design and deployment the logical starting point.

The goal was to create a professional, cohesive, and secure online environment for Kajatronics as shown in figure 3.1

Steps :

1. Establishing the Digital Real Estate: Server and Technology Stack Configuration

The very first action was to move from concept to a tangible, live environment.

- **The Groundwork:** I began by procuring a Premium hosting plan from **Hostinger** and registering the project's domain name. This step immediately established the live server environment that would host the application.
- **The Core Technologies:** On this new server, I configured the essential technology stack that would power the entire platform.
    - **PHP 8:** I selected PHP as the primary backend language. Its maturity, extensive documentation, and tight integration with MySQL make it an incredibly reliable and efficient choice for building the kind of dynamic, database-driven application this project required. It would be the workhorse handling all server-side logic.
    - **MySQL:** A MySQL database was created. This would serve as the central, secure vault for every piece of information the platform would handle, from a user's hashed password to every time-stamped sensor reading.

3.3.2 Objective 2: Crafting the Brand's Digital Identity - The Corporate Showcase

With the secure backend infrastructure in place, the next logical step was to build the platform's public face. This objective was focused on addressing the **Weak Digital Identity** gap by creating a professional, compelling, and fully responsive corporate website.

This was more than just an About Us page, it was engineered to be a powerful marketing tool, designed to build trust, showcase expertise, and convert visitors into potential clients.

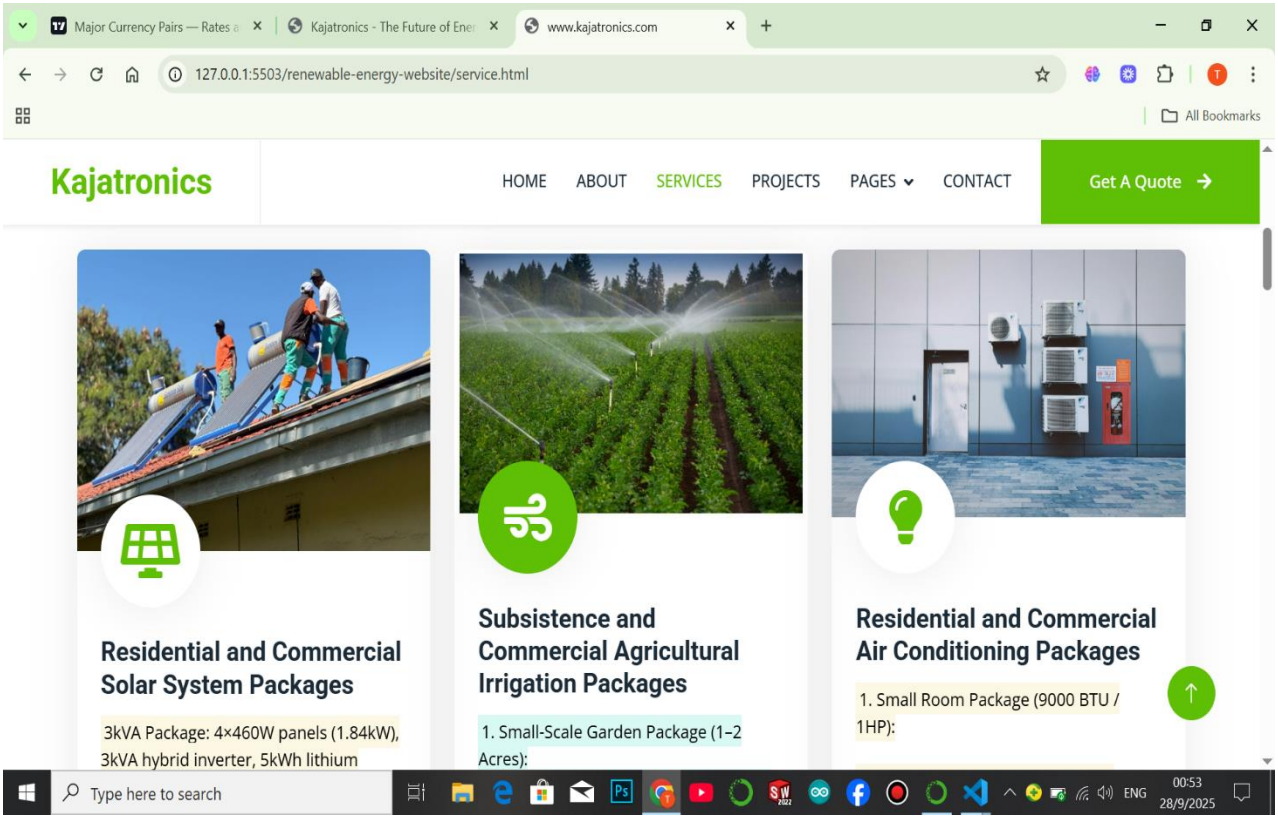


Figure 3.2 : Kajatronics website home page

Steps :

1. The Blueprint: Wireframing and Content Strategy

Before writing any code, I mapped out the user's journey. Then created wireframes - simple black-and-white layouts for the essential pages such as the homepage to make a strong first impression, a services page to detail their offerings, a projects page to provide social proof, and a contact page to generate leads as shown in figure 3.2

I then worked with Kajatronics to gather the high-quality images and compelling text that would populate this structure.

2. Building the Foundation: A Deep Dive into Responsive Frontend Development

The technical execution was a meticulous layering of modern frontend technologies, with the absolute priority being a flawless experience on any device.

- **A. The Structural Skeleton (HTML5):**

  The website's structure was built using semantic **HTML5**. I used tags like <header>, <nav>, <main>, <section>, and <article> to create a document that is not only logically structured for developers but is also highly optimized for search engine crawlers and accessibility tools.

  This clean structure is the foundation of a professional web presence

- **B. Mastering the Multi-Device Challenge (Bootstrap 5 and CSS3):**

  A website that is not mobile-friendly is a website that is losing customers. To solve the critical challenge of responsiveness, I built the entire website on the **Bootstrap 5** framework.

  - **The Mobile-First Grid:** Bootstrap's grid system was the key. For the project portfolio page, for example, I used Bootstrap's **card** components.
  - I then placed these cards within the grid system using classes like col-md-6 col-lg-4. This single line of code is incredibly powerful. It tells the browser that on a phone (small screen), make each project card take up the full width.

    On a tablet (medium screen), show them in a two-column grid. On a large desktop monitor, arrange them in a beautiful three-column grid." This intelligent, adaptive layout ensures the content is always perfectly presented, no matter how the user is viewing it.
  - **Brand-Centric Styling:** On top of the Bootstrap framework, I layered custom **CSS3** to infuse the site with the unique Kajatronics brand. I used CSS Flexbox to perfectly align elements within components and added subtle CSS transitions to links and buttons, which create a smooth, premium feel as the user interacts with the site.

- **C. The Interactive Polish (JavaScript):**

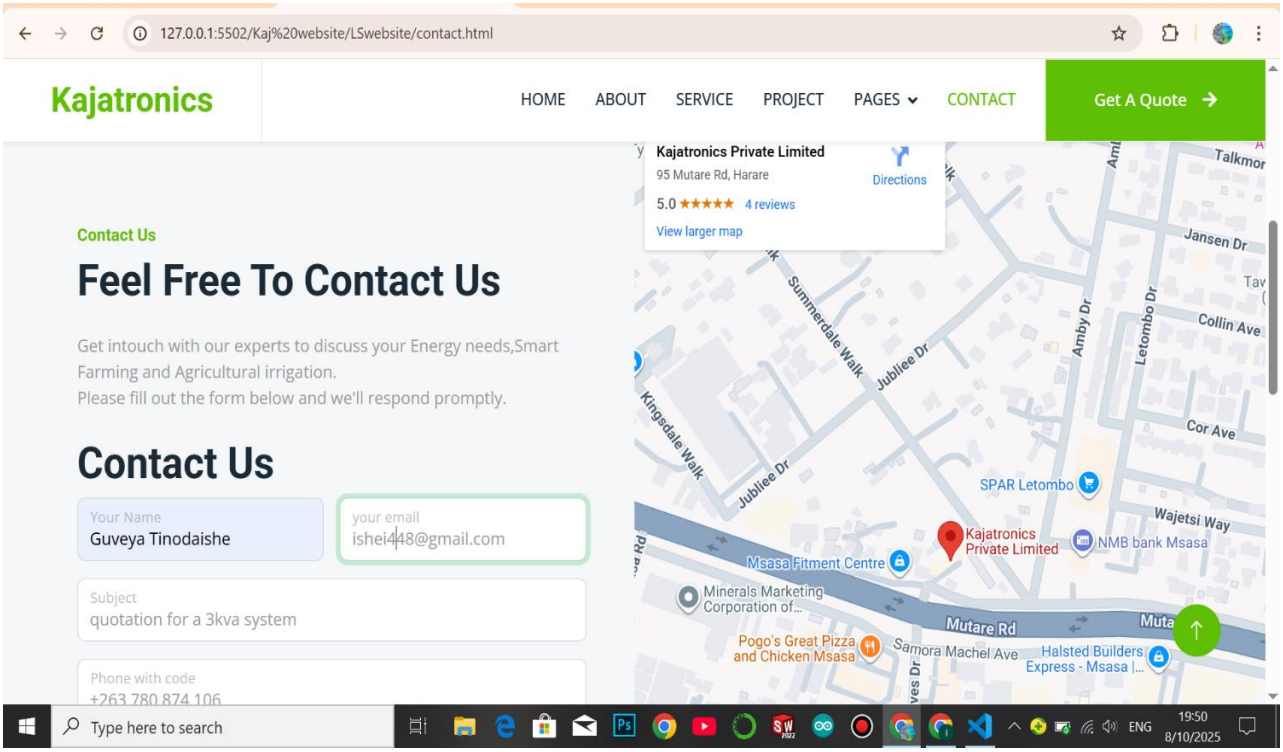


Figure 3.3 : shows the contact form and the location of Kajatronics Private Limited

To elevate the Website from a static brochure to an interactive experience as shown in figure 3.3 , I used vanilla **JavaScript**.

- **Form Validation:** The contact form provides instant, client-side validation.
- If a user enters an incorrectly formatted email address, a helpful message appears immediately, without needing to submit the form and wait for a server response. This small detail dramatically improves usability.

- **Dynamic Content:** An interactive filter was added to the projects page, allowing users to click buttons to sort the project cards by category (e.g., "Solar," "Automation") in real-time, creating a more engaging and user-driven experience.

Objective 3. Designing the Secure Airlocks: The User Portals

With the server environment ready, the next critical task was to build the secure entry points for different types of users. The platform needed to intelligently distinguish between a paying client and an administrative technician.

- **Database Schema for Access Control:**

  The design began in the database. I created a users table with essential columns that is id, username, email, password_hash, and, most importantly, user_role. This user_role field, which would contain either Client or Technician, is the fundamental mechanism that allows the system to grant different levels of access.

- **Constructing the Login Mechanism - A Deep Dive into Security:**

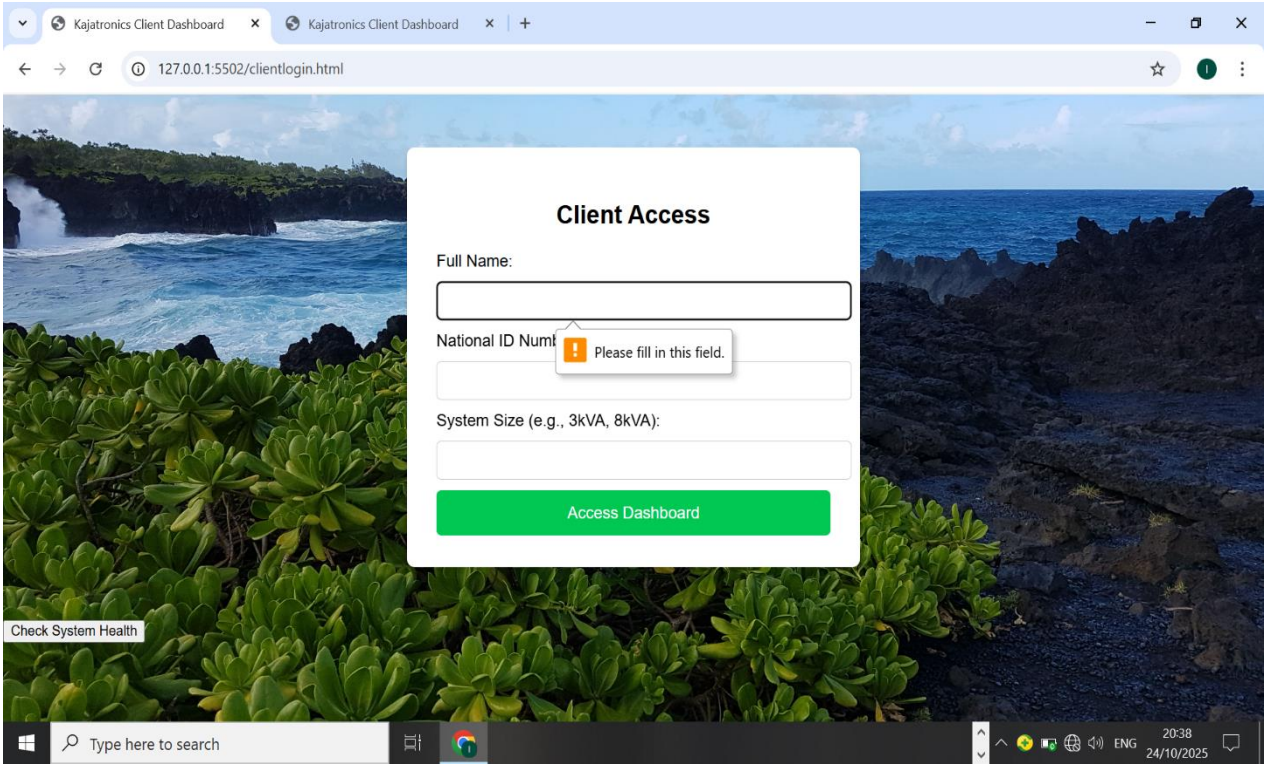


Figure 3.4: the client dashboard security mechanism

The user login system was engineered with a security-first mindset. This is not just a form. it's a multi-step security protocol.

1. **The User Interface (HTML5 & CSS3):** I built the clean, simple login forms using semantic **HTML5**. Custom **CSS3** was used to style them in line with the Kajatronics brand.
2. **The Backend Verification (PHP):** When a user submits their login details, the PHP backend springs into action.
    - It first **sanitizes** the inputs to prevent common security threats like SQL injection.
    - It then retrieves the record for the submitted username from the users table in the database.
    - **Crucially, it never handles plaintext passwords** : The user's submitted password is put through PHP's password_verify() function as shown n figure 3.4

      This function takes the submitted password and compares it against the secure hash stored in the password_hash column.

      This process is a one-way street , it's impossible to reverse the hash to get the original password, making it incredibly secure.

3. **Session Management:**

   If password_verify() returns true, a secure PHP session is initiated. The user's ID and, critically, their user_role are stored in the session.

   From this point on, for every page the user tries to access, a simple check is performed at the very top of the script: "Is the user logged in, and does their role match the required role for this page?" This ensures a client can never accidentally stumble upon the admin portal, and vice versa.

Objective 4: Embedding Intelligence - The Predictive Maintenance Engine

With a stable pipeline of real-time data flowing and being beautifully displayed, the project progressed to its most advanced and value-driven stage. The goal here was to elevate the platform from a simple monitoring tool into a proactive, intelligent advisor.

This meant building an engine that could automatically analyse the data, detect subtle patterns of impending failure, and provide clear, actionable alerts to the user.

Steps:

1. Choosing the Right Tool for the Job: The Analytical Engine

The heart of the predictive system needed to be powerful and flexible.

- **The Language (Python):** I chose **Python** as the language for this engine. While the main web application was built in HTML5, CSS3, BOOTSTRAP5 ,JAVASCRIPT AND PHP, Python is the undisputed king of data science and analysis.

  Its powerful libraries and clean syntax make it the perfect tool for the complex task of sifting through data to find meaningful patterns.

- **The Method (Rule-Based And Classification Algorithm):** For this initial implementation, I designed a **rule-based algorithm alongside a Classification model**. Instead of a complex "black-box" machine learning model, this approach uses a set of clear, human-understandable "IF-THEN" rules. This was a deliberate choice to ensure the system's recommendations were transparent, easy to debug, and directly based on the expert knowledge of field technicians.

2. The Automation Core: Scheduling the Analysis

A predictive system is only useful if it runs continuously. To achieve this, I used a standard Linux server tool called a **cron job**.

- **The Setup:** On the Hostinger server, I configured a cron job to automatically execute the Python script once every hour. This simple command ensures that, without any human intervention, the entire fleet of client systems is analyzed 24 times a day, 7 days a week.

3. The Analytical Workflow: How the Engine "Thinks"

Every hour, when the cron job awakens the Python script, it performs a methodical, three-step process:

1. **Fetch:** The script first connects to the **MySQL database** and retrieves all the new sensor data that has arrived since its last run.
2. **Analyze:** It then loops through the data for each individual client system and applies its set of pre-defined rules. This is where the intelligence lies. It asks critical questions of the data:
    - **Efficiency Rule:** "Is this solar panel's average power output over the last 30 days more than 5% lower than the average of the 90 days before that? If so, it might need cleaning."
    - **Temperature Rule:** "Has this battery's temperature exceeded the manufacturer's recommended peak operating temperature of 45°C in the last 24 hours? If so, there might be a ventilation problem."
    - **Overload Rule:** "Is this inverter consistently running at over 90% of its rated load for more than an hour each day? If so, the client's energy usage may have increased, and they risk premature failure."
3. **Act:** If any of these rules are triggered, the script's final job is to act. It generates a clear, human-readable alert message and inserts it as a new row into the dedicated alerts table in the database.

4. Closing the Loop: Presenting Wisdom to the User

The final step was to make this newfound intelligence visible.

- The existing Client and Admin dashboards, already programmed to display data from the database as shown in figure 4.8, will updated with a new module. This "Maintenance Alerts &

Recommendations" section was coded to do one simple thing: display the contents of the alerts table for that specific user.

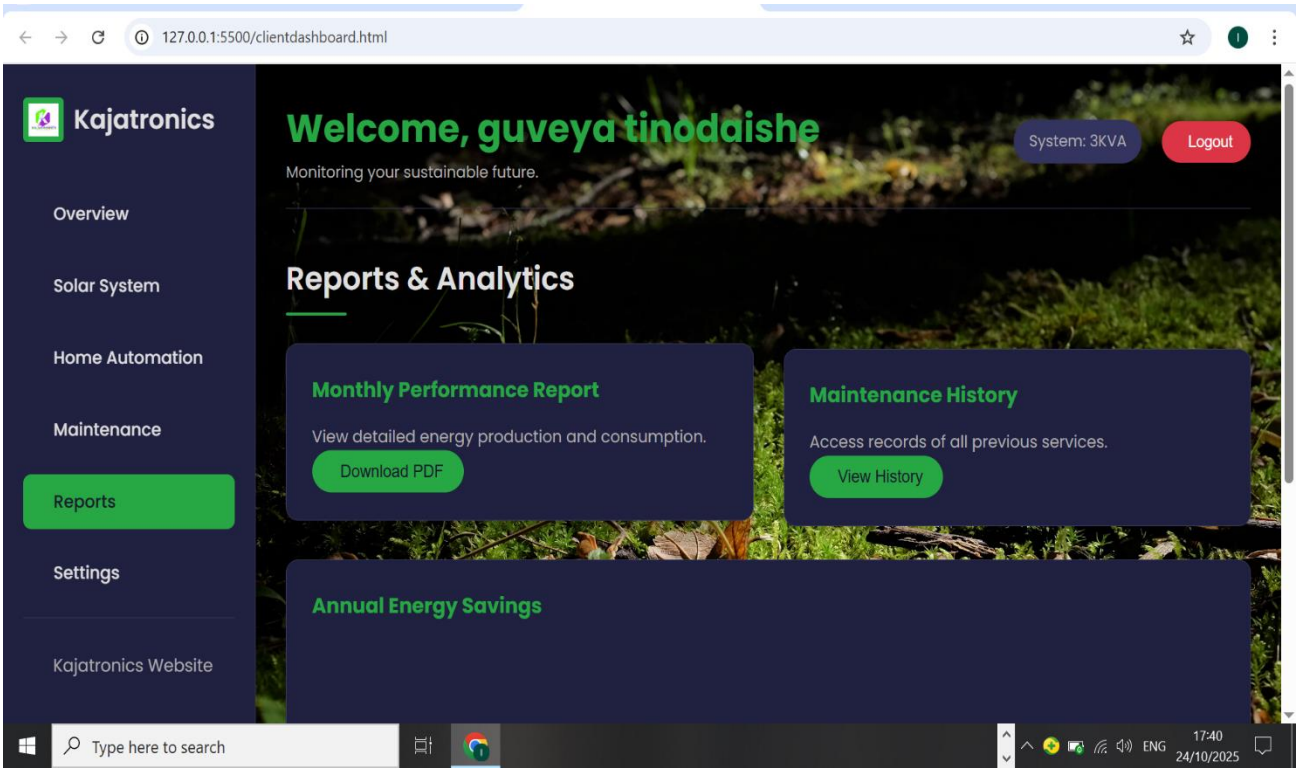


Figure 3.5: predictive maintenance dashboard (front-end layout)

Objective 5: Forging the Unbreakable Link - The Autonomous IoT Data Acquisition Unit

With the intelligent cloud platform fully engineered, the final and most fundamental objective was to create the physical source of its truth: a rugged, reliable, and fully autonomous IoT Data Acquisition Unit. This is the device that lives in the field, at the client's site, serving as the sensory organ of the entire "LivingStone Connect" ecosystem.

The core engineering challenge was to design a unit that could directly interface with a complex solar power system (inverter, battery, and PV panels), operate continuously, and transmit its vital data back to the server automatically, without relying on the client's Wi-Fi network.

Steps:

1. The Brains of the Operation: Microcontroller Selection and Rationale

The heart of the unit needed to be a microcontroller that was not only capable but also exceptionally reliable and power-efficient.

- **The Choice (Arduino Uno):** I selected the **Arduino Uno** for this critical role. While other options exist, the Arduino platform is renowned for its stability in harsh environments and its extremely low power consumption. In a system that would be powered by the very solar setup it monitors, every milliwatt of energy saved is crucial. Its simplicity is its greatest strength, ensuring it performs its dedicated task of data collection and transmission with unwavering reliability, free from the complexities and potential failure points of a full operating system.

2. Designing the Sensory Nerves: Sensor Selection and Integration

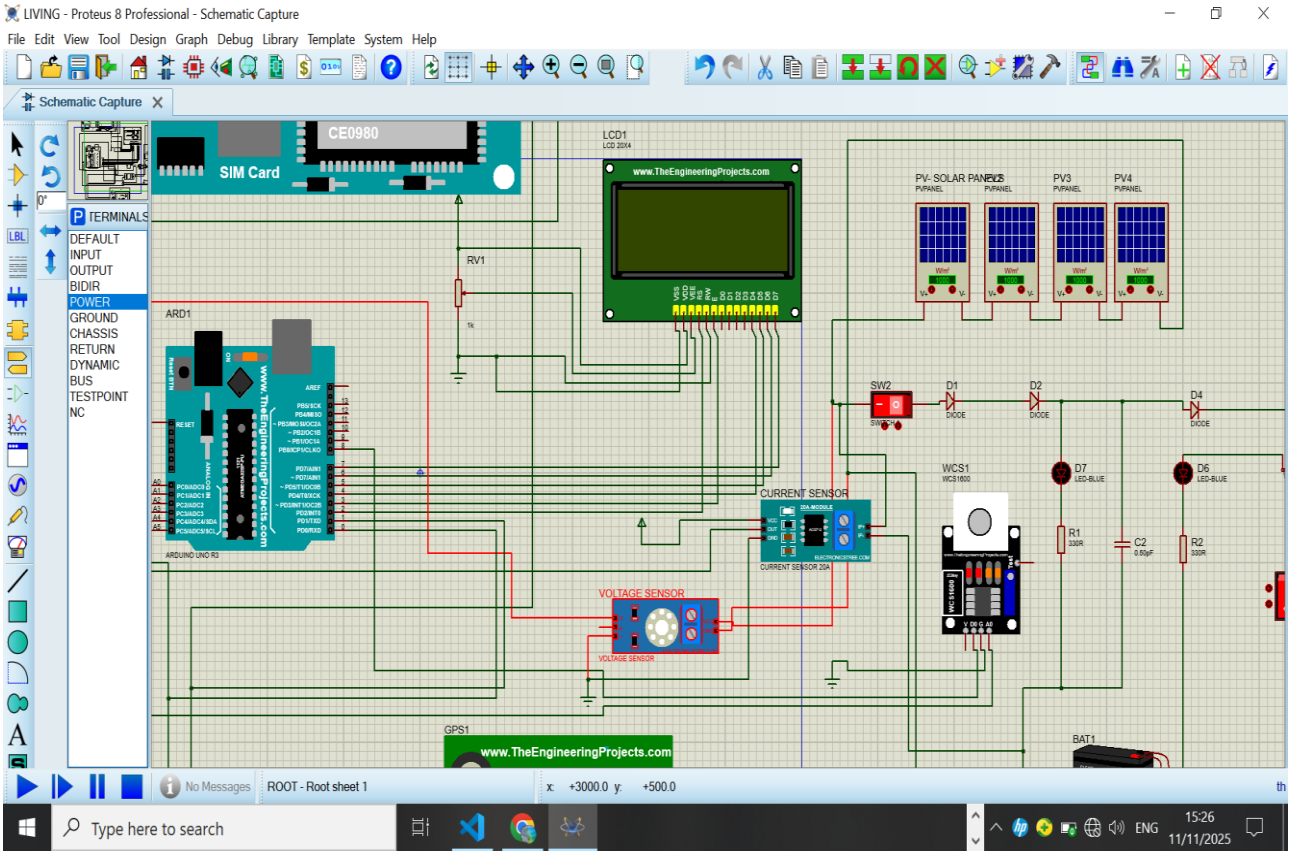
To get a complete health profile of the solar system, a carefully curated suite of sensors was required to act as the unit's "nerves," tapping directly into the vital signs of the equipment.

- **A. Monitoring the Heart (The Hybrid Inverter):**
    - **The Sensor (ZMPT101B Voltage Sensor & ACS712 Current Sensor):**

      The health of the inverter is best understood by its output. A **ZMPT101B** module was integrated to safely and accurately measure the AC voltage it produces. Simultaneously, a non-invasive **ACS712 Hall-effect current sensor** was placed on the main output line.
    - **How It Was Used:** These two sensors provide the raw data needed for the most critical calculation: Power (Watts) = Voltage (Volts) × Current (Amps).

      By constantly monitoring the power output, the platform can track the system's overall performance and efficiency.

- **B. Checking the Lungs (The PV Panels):**
    - **The Sensor (Another ACS712 Current Sensor):** To understand how well the solar panels are performing, a second **ACS712 current sensor** was placed on the DC line running from the PV panels to the inverter/charge controller.
    - **How It Was Used:** By measuring the current coming directly from the panels, the system can determine how much energy is being generated. When compared with the inverter's output, it can help diagnose issues like dirty panels or wiring faults that reduce generation efficiency.



- **C. Taking the Temperature (The Lithium Battery):**
    - **The Sensor (DS18B20 Waterproof Temperature Probe):** Lithium batteries are highly sensitive to temperature. A rugged, waterproof **DS18B20** digital temperature probe was physically attached to the casing of the battery bank.

- **How It Was Used:** This sensor provides a direct, real-time temperature reading of the battery. This data is critical for the predictive engine, as a consistent rise in temperature is a primary early warning sign of cell degradation or a potential thermal runaway event.

The complete wiring of these sensors to the Arduino's analog and digital input pins was first designed and validated in the **Proteus Design Suite** to ensure electronic compatibility and prevent errors before physical assembly.

3. The Voice of the Unit: Autonomous Communication Technology

The most critical requirement for this unit was its ability to communicate independently, without relying on the client's potentially unreliable or non-existent Wi-Fi.

- **The Technology (GSM/GPRS):** I chose to integrate a **SIM800L GSM/GPRS module**. This tiny component is essentially a miniature cell phone for electronics.
- **Why It Was Used:** By inserting a standard SIM card (from a provider like Econet or NetOne), the unit gains the ability to connect to the cellular data network (GPRS), which has near-ubiquitous coverage across Zimbabwe. This decision completely decouples the monitoring system from the client's local network, guaranteeing that as long as there is a cell signal, the data can get through. This is the key to creating a truly autonomous and universally deployable solution.

NB: for the purpose of the academic project timeline, the hardware integration of sensors to solar system and their simulation wasn't documented as explained in the Project Proposal. However, it is being done separately in relation to the demands of Kajatronics Private Limited .

The main focus was directed to the heart of the entire system (the multipage coorparate website and intelligent client dashboard for predictive maintenance)

4. The Soul of the Machine: Firmware Logic and the Data Transmission Cycle

The final step was to write the firmware in **C++** that would orchestrate this entire process. The code, uploaded to the Arduino, is an elegant and efficient loop that runs indefinitely.

- **The Chronological Workflow:**

1. **Awake:** The microcontroller wakes from its low-power sleep state.
2. **Sense:** It sequentially queries each of the integrated sensors—reading the voltage and current from the inverter, the current from the PV panels, and the temperature from the battery.
3. **Package:** It takes these raw readings and meticulously formats them into a lightweight, efficient **JSON string**. This structured text format is the universal language of modern web APIs. A sample payload would look like: {"inverter_v":230.7, "inverter_a":3.1, "panel_a":8.2, "batt_t":29.5}.
4. **Connect & Transmit:** The firmware then commands the **SIM800L GSM module** to wake up and connect to the GPRS network. Once connected, it opens a secure HTTP POST request and sends the JSON string directly to the predefined endpoint on the "LivingStone Connect" server.
5. Confirm and Sleep:

   The firmware waits for a success confirmation (HTTP 200 OK) from the server. Once received, it immediately commands itself and the GSM module to enter a deep-sleep mode.

- **Resilience and Autonomy:**

  This entire cycle repeats automatically every fifteen minutes. If the cellular network is temporarily unavailable, the firmware is programmed to store the last failed reading and attempt to send both it and the current reading on the next successful connection, ensuring no data is lost. This complete autonomy is what makes the hardware a truly "set it and forget it" solution, reliably feeding the cloud platform with the vital data it needs to function, without ever requiring a Wi-Fi password or any intervention from the client.

5 Solution Optimisation: A Holistic Approach to Refinement

A project of this complexity is like a living organism : optimising one part can have profound effects on the whole. The journey from a functional first version of **LivingStone Connect** to the final, polished product was a holistic process of optimisation.

This wasn't just about tweaking code , it was about re-evaluating the entire user experience, from the moment a potential customer lands on the website to the second a client receives a critical maintenance alert.

Every component from the public website, the client dashboard, the backend engine, and the physical hardware was placed under the microscope and refined for performance, efficiency, and user-centricity.

1. Iterating on the Multi-Page Website Based on Feedback:

- **The Initial Design:**

  The first version of the Kajatronics corporate website was professional and informative, successfully showcasing the company's services.

- **The Feedback:**

  During a User Acceptance Testing (UAT) session, I observed how potential users interacted with the site. A key piece of feedback was that while the Home and projects page was impressive, it was just a static gallery. Users expressed a desire to understand *how* Kajatronics' technology solved real-world problems.

- **The Optimisation (Refining for User Engagement):**

  Based on this, I iterated on the website's design. Instead of just a gallery, the home and projects page was transformed into a case study hub.

  Each project now included not only images but also a short narrative detailing the client's initial problem, the Kajatronics solution.

2. Optimising the Client Dashboard for Peak Performance and Clarity:

- **The Initial Design:** The first client dashboard prototype successfully displayed live data and alerts.

- **The Performance Bottleneck:** During testing with simulated data from 50+ clients, I noticed a critical performance issue. The dashboard's initial load time began to slow down considerably. The problem was that the backend was trying to fetch and process large amounts of historical data every time the page loaded.
- **The Optimisation (Backend & Frontend Synergy):**
    - **Backend Refinement:** I re-engineered the backend PHP API. Instead of sending a huge chunk of historical data on the initial load, the API was split.

      The initial page load now only sends the most recent data needed for the "live" view. Historical data is now only fetched when the user specifically requests it by clicking on a "View History" button.

      I also implemented **database indexing** on the sensor_logs table, which dramatically reduced the time it took to query this historical data, making the "View History" feature feel instantaneous.
    - **Frontend Refinement:** On the client's dashboard itself, I implemented **lazy loading** for the charts using JavaScript. The chart library is now only initialized and rendered *after* the rest of the page content is visible. This makes the page *feel* much faster to the user, as they can immediately see their system status and alerts without having to wait for the complex chart to draw.

5B. Selecting an Optimum Solution: Strategic Decisions for a Cohesive Ecosystem

The final architecture of the LivingStone Connect platform is the result of a series of deliberate, strategic decisions made at critical junctures.

For every core component of the ecosystem, from the public website's underlying technology to the hardware's communication protocol, alternative paths were evaluated. The optimum solution was consistently defined as the one that offered the best synthesis of **Performance**, **Cost**, **Scalability**, and **Reliability** within the project's specific context.

Ecosystem Component	Alternative Option	Selected Solution and Justification
Public Website Platform	WordPress (Template-based CMS)	Custom HTML/CSS/JS: This path was chosen to prioritize peak performance, enhanced security, and complete design freedom over the rapid but often bloated and vulnerable nature of template-based systems.
Client Dashboard Framework	Heavy JS Framework (e.g., React)	Vanilla JavaScript with Bootstrap 5: This lightweight approach was selected to optimize for development speed and long-term maintainability, providing all necessary functionality without the high complexity of heavier frameworks.
Hardware Communication	Wi-Fi Module	GSM/GPRS Module: This was a critical decision for platform reliability. GSM guarantees an autonomous and consistent data link, completely

		independent of the client's often unreliable or non-existent Wi-Fi, making it the only truly viable solution.

Table 3.1: The table presents a comprehensive evaluation of the key design choices that shaped the entire platform, from its public face to its innermost workings.

6 Bill of Materials (BOM): The Recipe for a Scalable Solution

The final piece of the design process is the translation of the optimised architecture into a practical, cost-conscious list of materials.

This Bill of Materials (BOM) was developed with a key project goal in mind: to achieve a professional-grade, reliable hardware unit for a total cost of USD $100.

category	component	description	quantity	Unit cost (usd)	Total cost (usd)
Core logic and comms	Arduino uno r3	The central microcontroller processing all sensor data. Chosen for its proven reliability and extensive support.	1	$25-00	$25-00
	SIM800L GSM/GPRS	Provides autonomous	1	$15.00	$15.00

	Module (with breakout board and antenna)	cellular data connectivity. A higher-grade module ensures better signal strength and stability			
Sensor Suite	**ZMPT101B Voltage Sensor Module**	Safely measures the inverter's AC voltage output.	1	$10.00	$10.00
	ACS712 Hall Effect Current Sensor (60A Module)	Measures the AC current from the inverter. Sourced for accuracy and durability under continuous load.	1	$15.00	$10.00
	DS18B20 Waterproof Temperature Probe	Industrial-grade probe for accurately monitoring battery temperature in harsh conditions.	1	$10-00	$10-00

Supporting Materials	**IP65 Weatherproof Enclosure**	A durable, sealed plastic box to protect electronics from dust, moisture, and insects.	1	$7-00	$5-00
	5V DC Buck Converter	A high-efficiency regulator to step down power for the electronics, minimizing energy loss.	1	$5-00	$5-00
	Prototyping Board & Wiring Components	High-quality PCB, screw terminals, connectors, and wiring for a professional and durable assembly.	1	$10-00	$10-00
Subtotal					**$95-00**
Contingency (15%)	Allocated for unforeseen needs, shipping costs, or component price fluctuations	$10-00	1	$5-00	**$5**-00

	during procurement.				
Final Estimated Cost Per Unit					**$100-00**

Table 3.2: The table provides a detailed breakdown of the components required to construct one IoT Data Acquisition Unit.

This cost-effective design is a cornerstone of the project's viability. It proves that a sophisticated, autonomous monitoring system can be produced at a price point that makes it accessible and highly scalable for deployment across Kajatronics' entire client base, ensuring the LivingStone Connect platform is not just a technological success, but a sustainable business solution.

Chapter 4

4.0 Introduction

This chapter presents the tangible outcomes of the LivingStone Connect project, transitioning from the theoretical design phase to a functional, deployed system. Each result is presented with visual evidence and discussed in the context of the initial project objectives.

The chapter will demonstrate the successful creation of a professional corporate web presence designed to build trust and generate leads, followed by an in-depth look at the secure, role-based portals that form the gateway to the platform's core monitoring functionality.

Finally, it will showcase the client dashboard, the hub of real-time data and actionable insights, thereby validating the project's success in its entirety.

4.1 Result 1: The Corporate Multi-Page Website

The first major objective was to resolve the "Weak Digital Identity" by creating a professional, multi-functional corporate website. The successfully deployed platform serves as the digital front door for Kajatronics, engineered not only to inform but also to build credibility, showcase expertise, and convert visitors into clients.

- **A Professional Showcase of Services and Expertise**

The website immediately establishes Kajatronics as a serious and capable player in the renewable energy sector. The homepage, as shown in **Figure 4.1**, features a clean, modern layout with high-quality imagery that clearly communicates the company's core services. This professional presentation is critical for making a strong first impression and conveying a message of quality and reliability to potential customers exploring their options.

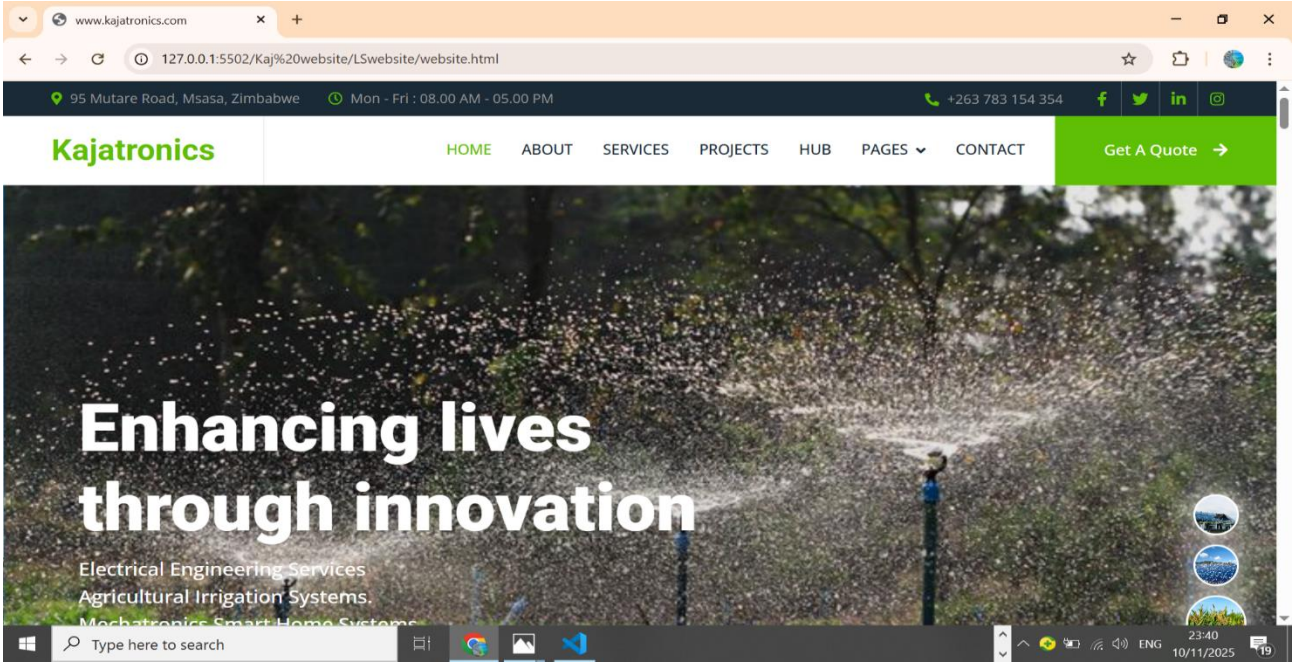


Figure 4.1: The website's professional presentation of home page top part.

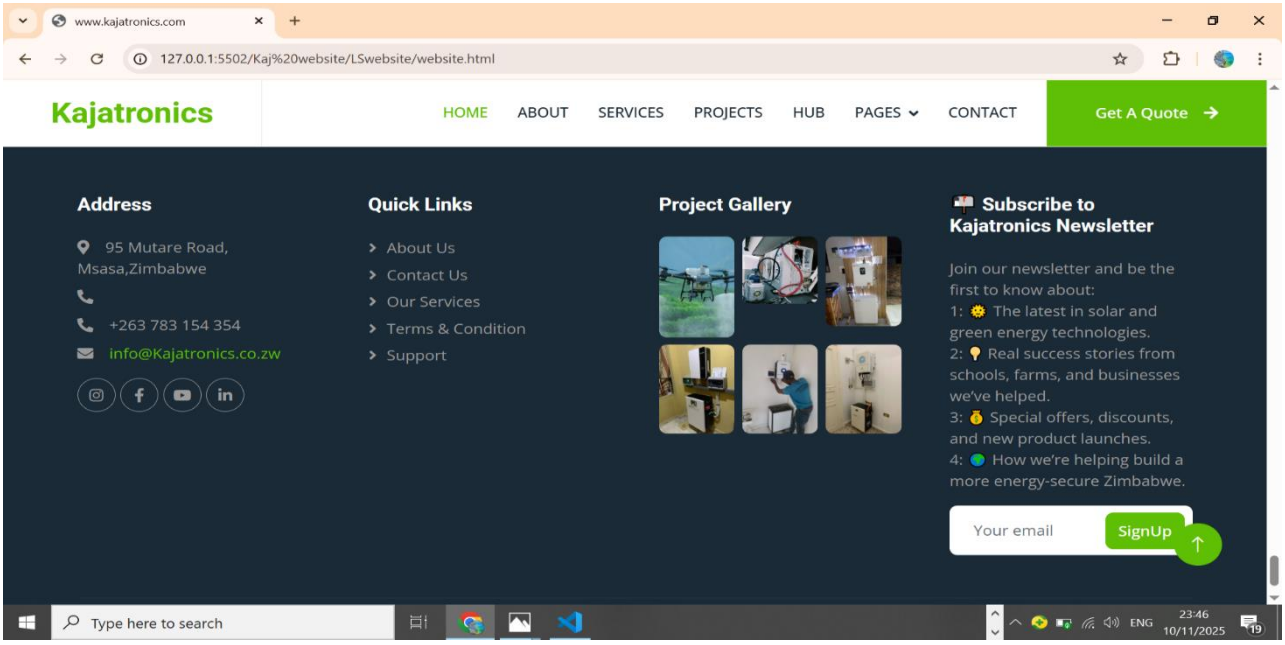


Figure 4.1.2: The website's professional presentation of home page bottom part.

- **Building Trust Through a Visual Project Portfolio**

To move beyond claims and provide concrete proof of capability, a comprehensive "Projects" section was developed. This section, depicted in **Figure 4.2**, serves as a dynamic visual portfolio of successfully completed installations.

By showcasing a variety of real-world projects, from residential battery backups to larger-scale systems, the website provides tangible evidence of Kajatronics' experience and the quality of their workmanship, which is fundamental to building trust with prospective clients.

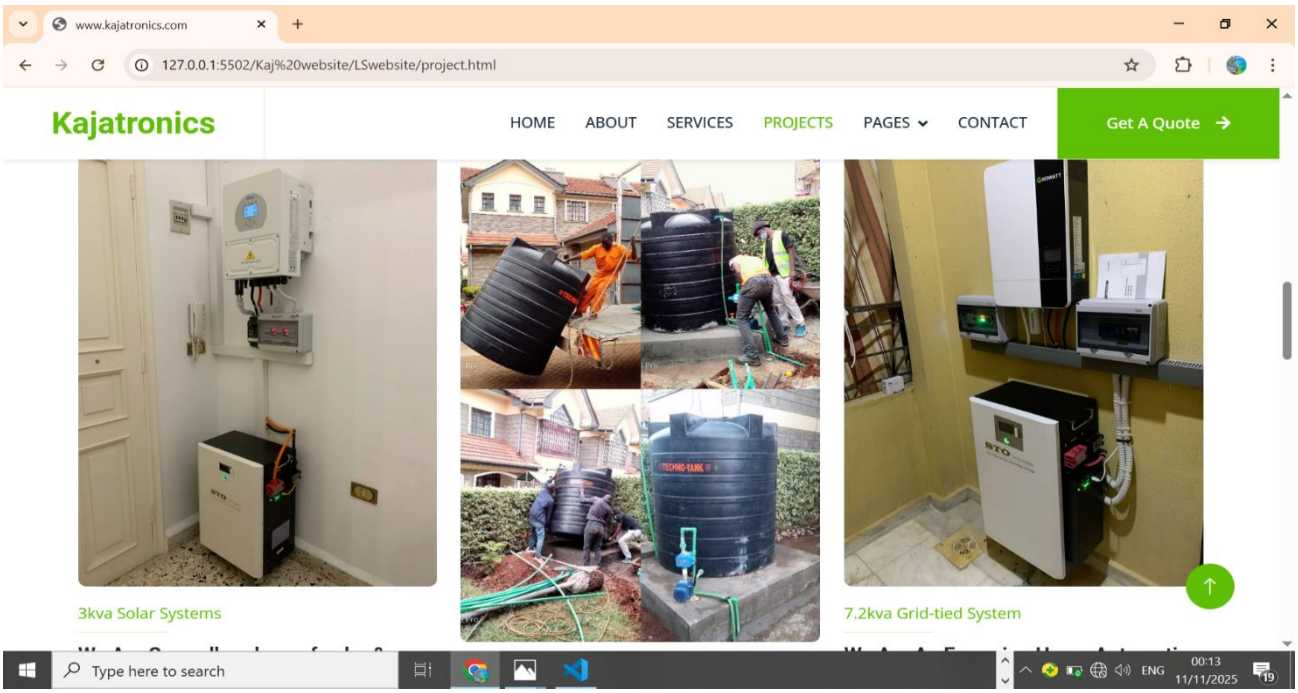


Figure 4.2: The project gallery showcasing diverse, real-world installations as evidence of expertise.

- **Active Lead Generation and Client Engagement**

The website was engineered to be an active business development tool. An intuitive "Get A Free Quote" form, shown in **Figure 4.3**, is strategically placed to make it easy for interested visitors to take the next step. This feature transforms the website from a passive source of information into an active lead generation engine, creating a direct pipeline for the Kajatronics sales team and streamlining the initial client engagement process.

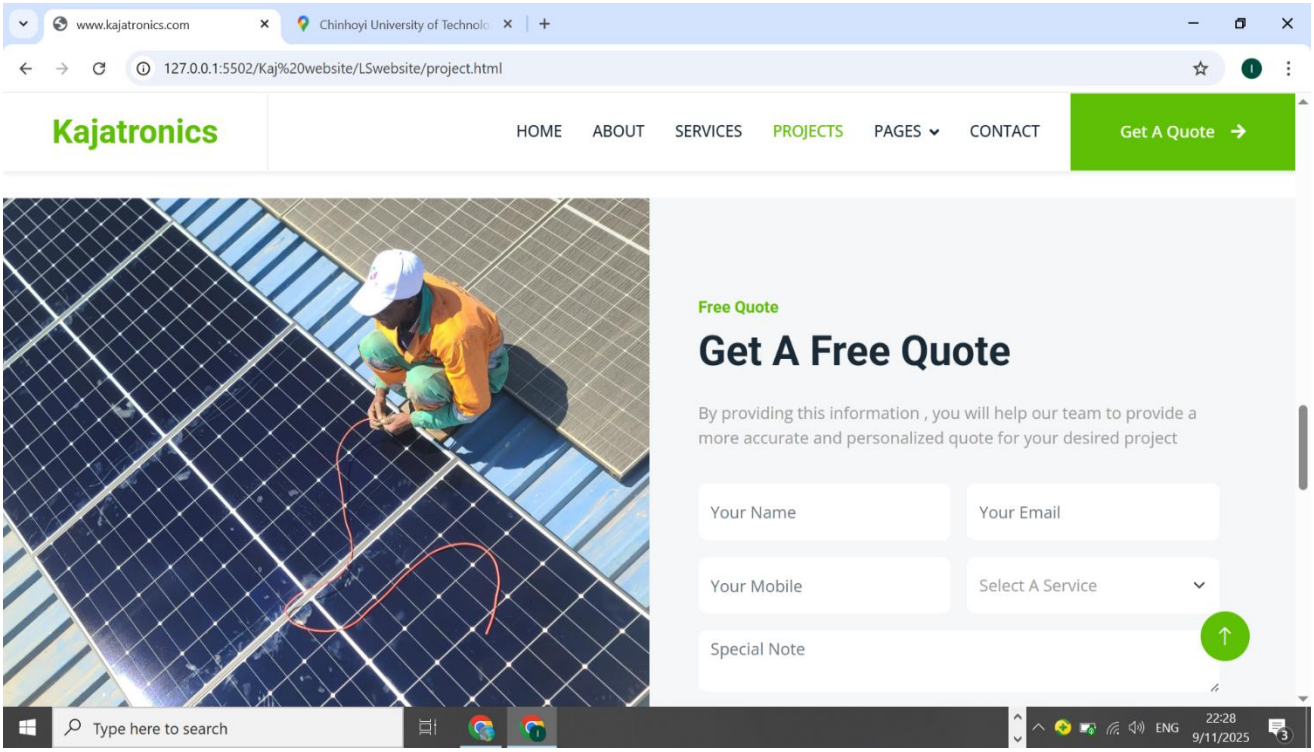


Figure 4.3: The integrated "Get A Free Quote" form for active lead generation.

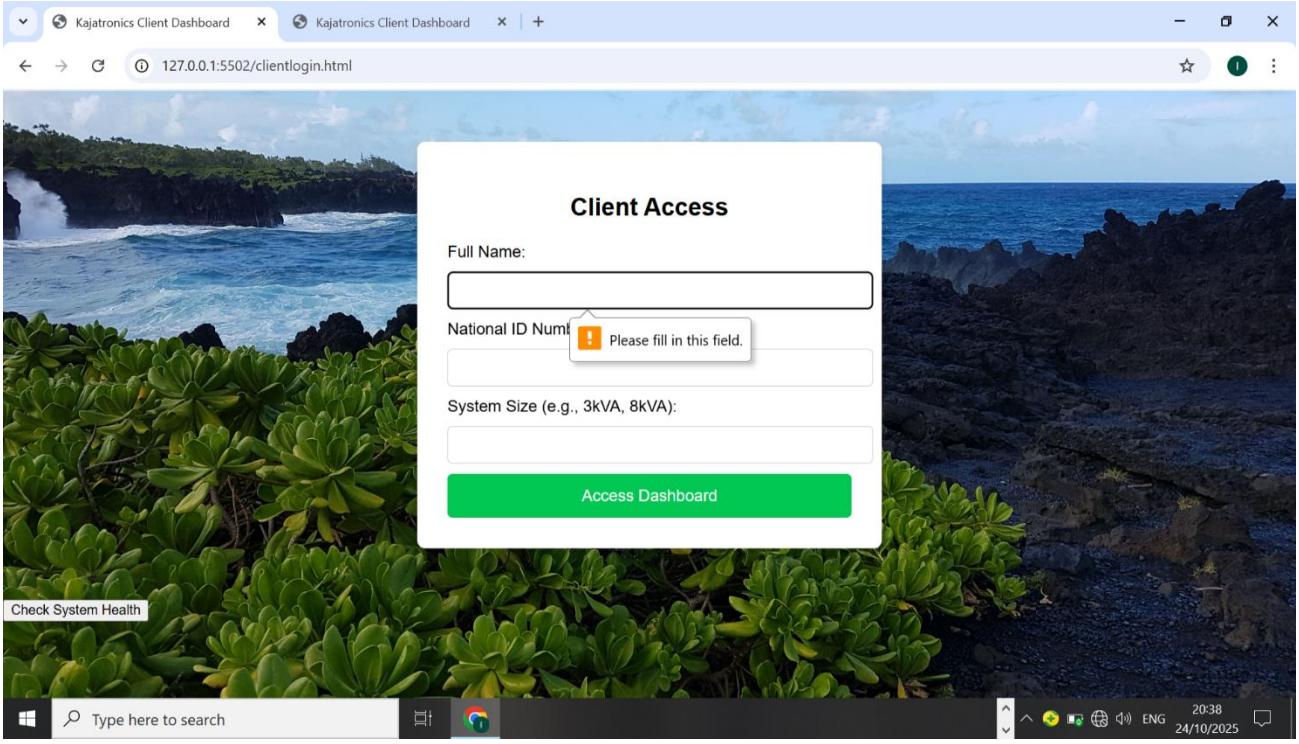
4.2 Result 2: Secure User Authentication and Verification

A cornerstone of the platform's design is its ability to provide secure, role-based access to sensitive system data. This objective was successfully achieved through the implementation of a robust authentication system that distinguishes between clients and administrative staff, ensuring data integrity and privacy.

- **Role-Based, Secure Access Portals**

The system features distinct and separate login portals for clients and company engineers, forming the first and most critical layer of security. **Figure 4 .4** shows the "Client Access" portal, which includes essential front-end validation, evidenced by the "Please fill in this field" prompt.

This ensures a user-friendly experience while gathering the necessary information. **Figure 4.4** displays the parallel "Engineers Access" portal. This strict separation of entry points ensures that users can only access the sections and data relevant to their specific role.



. **Figure 4.4** displays the parallel "client Access" portal

4.3 Result 3: The Client Monitoring Dashboard

Once authenticated, the user enters the heart of the LivingStone Connect platform: the client dashboard. This is where the project's core objective of providing real-time monitoring is realized, transforming raw sensor data into an easily understandable and actionable format for the end-user.

- **At a Glance System Overview**

The main dashboard screen provides an intuitive System Overview (**Figure 4.5**) that gives the client an immediate understanding of their system's health and performance. Key metrics are displayed in clear, bold widgets: **Overall Status** provides a simple Online or Offline health check, **Today's Energy Production** quantifies the system's daily output, and **Current Power Output** offers a live feed of performance. This design philosophy prevents information overload and empowers clients with the most critical data at first glance

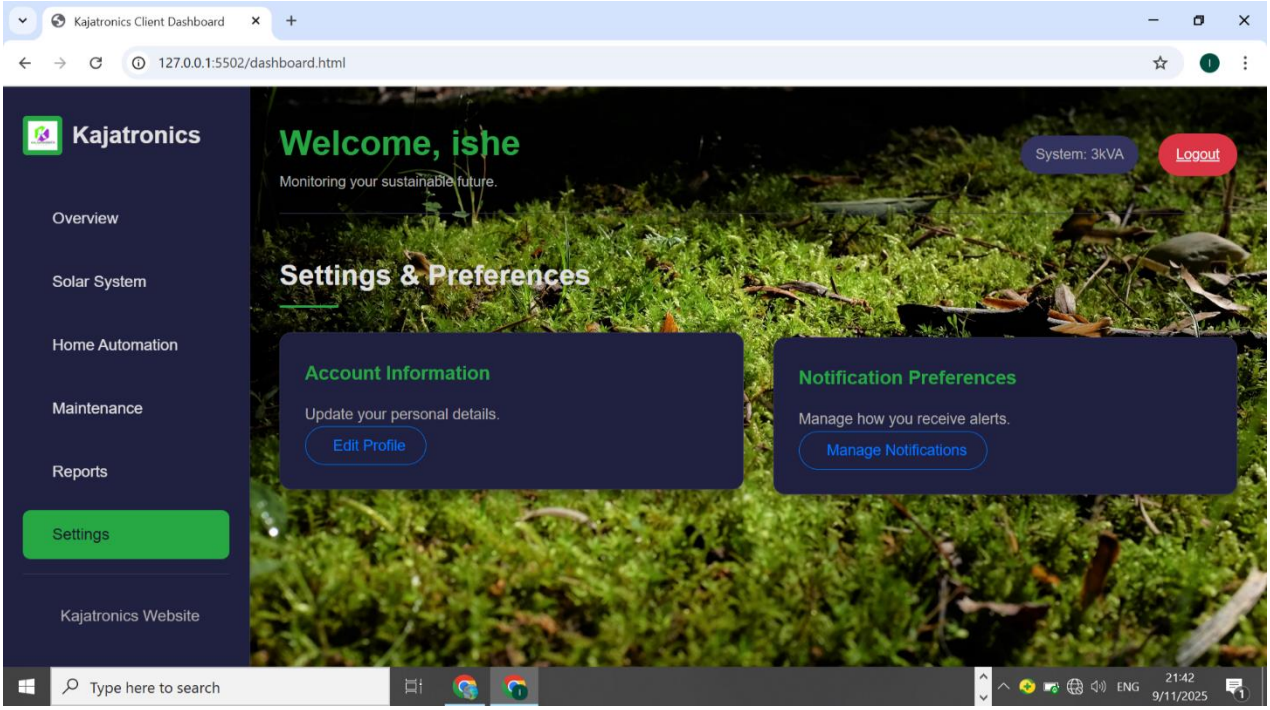


Figure 4.5: The client dashboard's main "Settings" with key real-time metrics

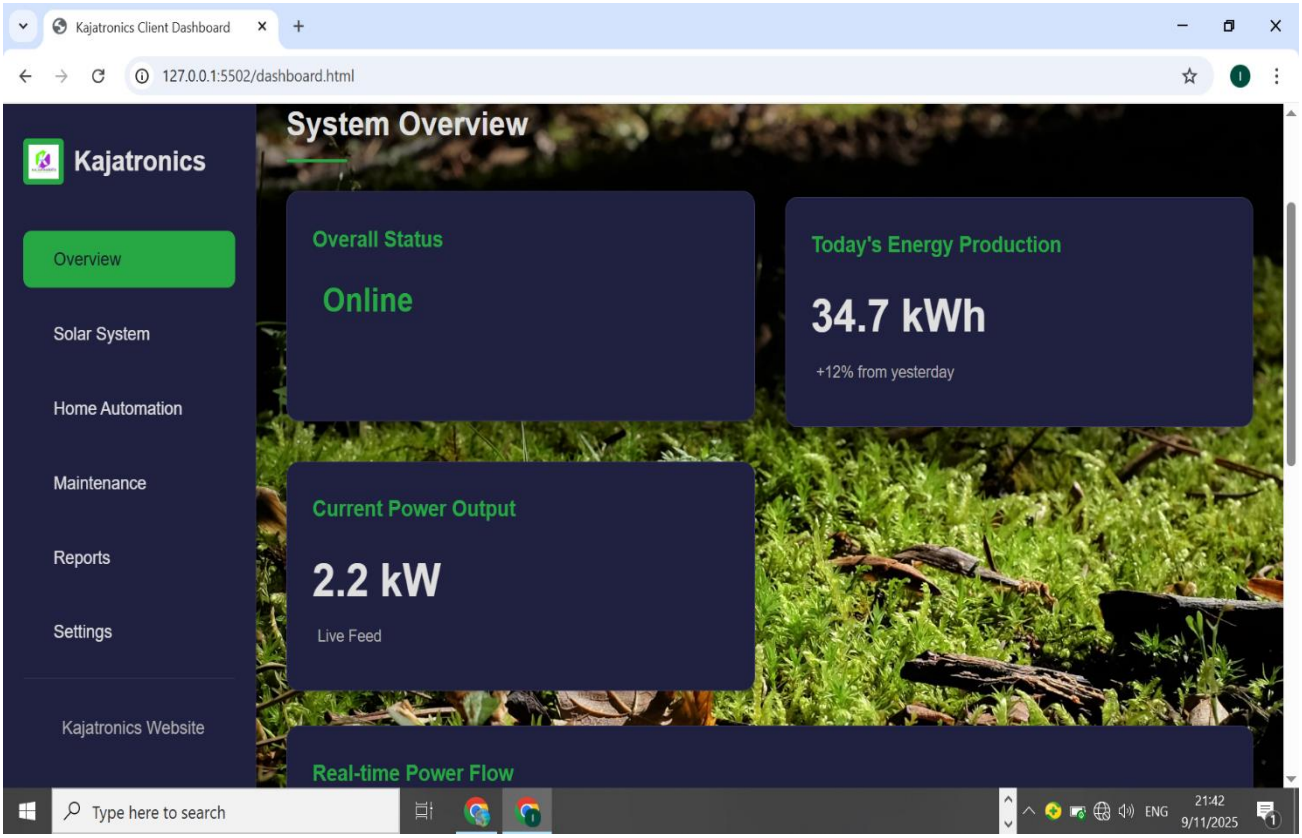


Figure 4.6: The client dashboard's main "System Overview" with key real-time metrics

- **Deep Dive into Reports and System Details**

The platform empowers clients to look beyond live data. The "Reports & Analytics" section, shown in **Figure 4.6**, allows users to access historical data, download detailed monthly performance reports, and review their complete maintenance history.

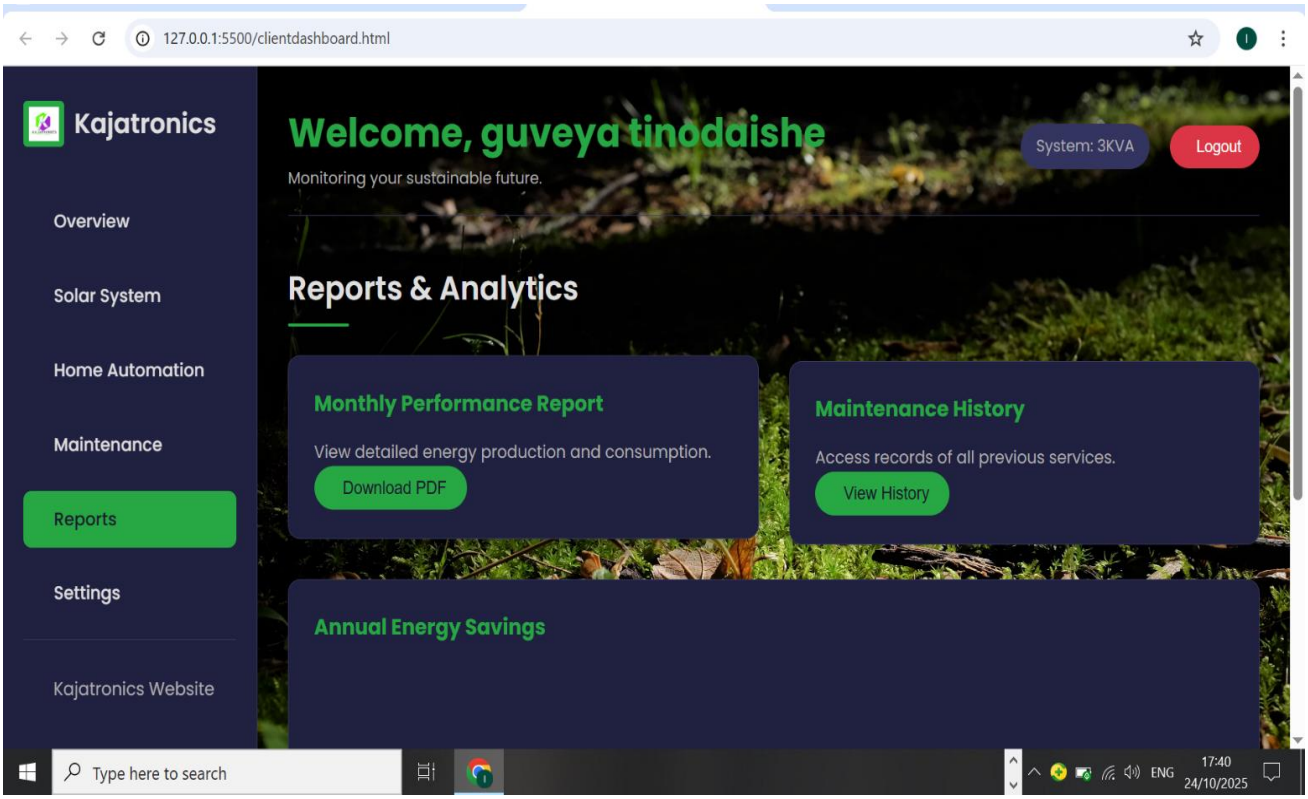


Figure 4.7: The client dashboard's main "Reports" with key real-time metrics

Furthermore, the platform provides granular **System Details** (**Figure 4.9**), including the installation date and scheduled maintenance based on sensor data from installed system as illustrated in **figure 4.8** and **figure 4.9** , fostering complete transparency and building long-term customer trust

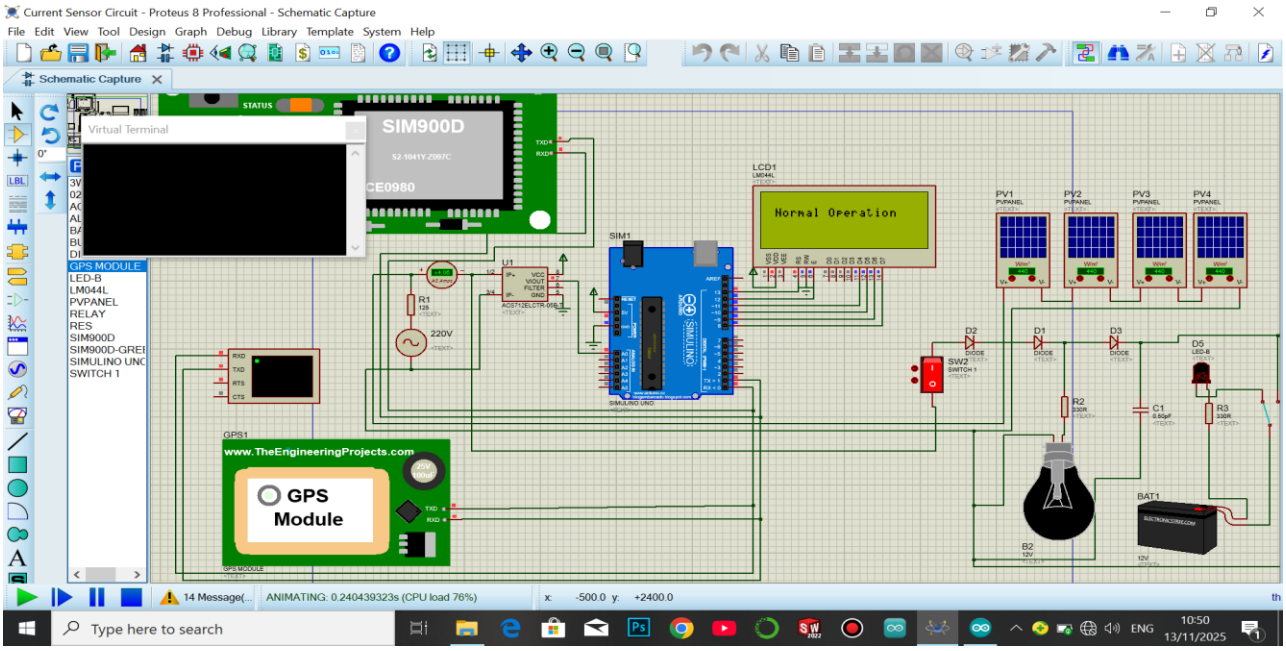


Figure 4.8: Hardware based system side(installed Arduino based system monitoring the 3.2 KW solar system.

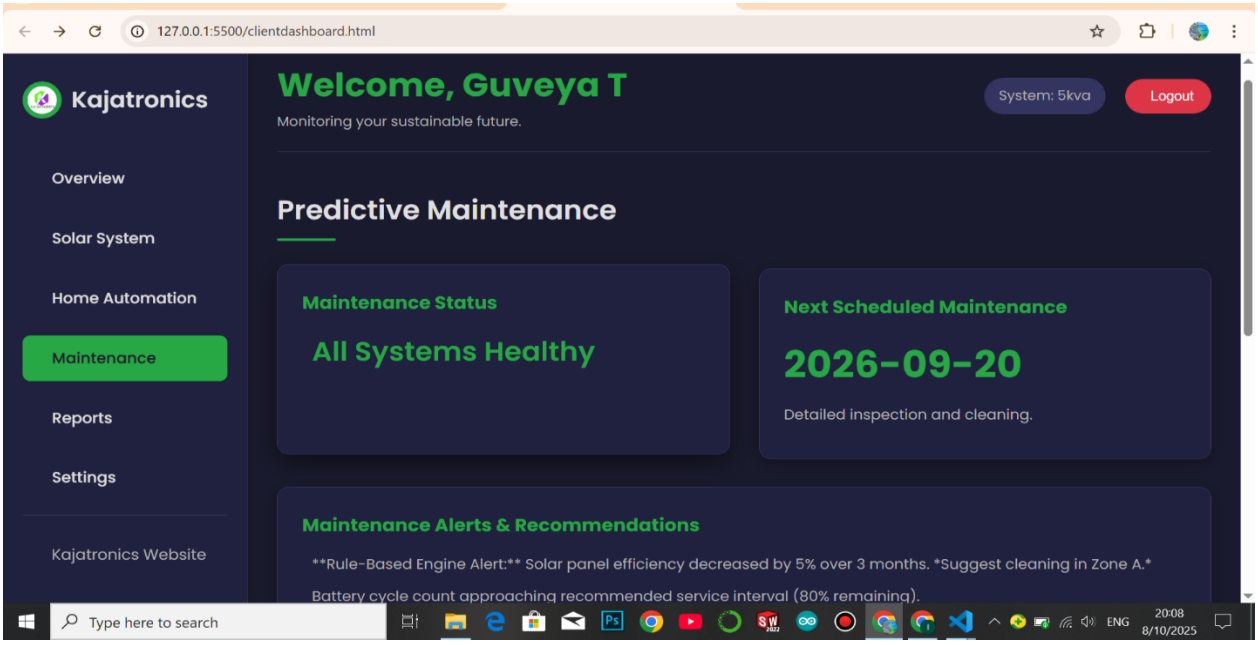


Figure 4.9: The client dashboard's main "System Health Dashboard" with key real-time metrics

4.4 Results accuracy based on the Logistic Regression Machine Learning model.

1. From the plot of the accuracy,the model could probably be trained a little more as the trend for accuracy on both datasets is still rising for the last few epochs. the model has not yet over-learned the training dataset, showing comparable skill on both datasets as shown in figure 4.10

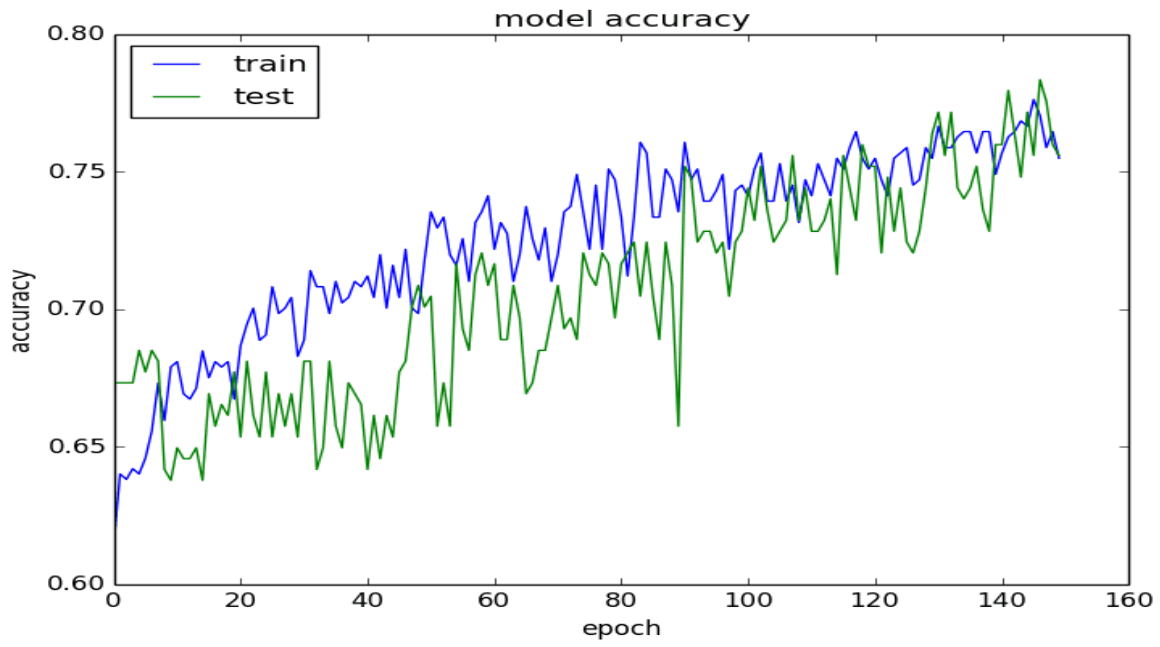


Figure 4.10: Plot of model accuracy on train and validation datasets

2. From the plot of the loss, the model has comparable performance on both train and validation datasets (labeled test). If these parallel plots start to depart consistently, it might be a sign to stop training at an earlier epoch as shown in figure 4.411

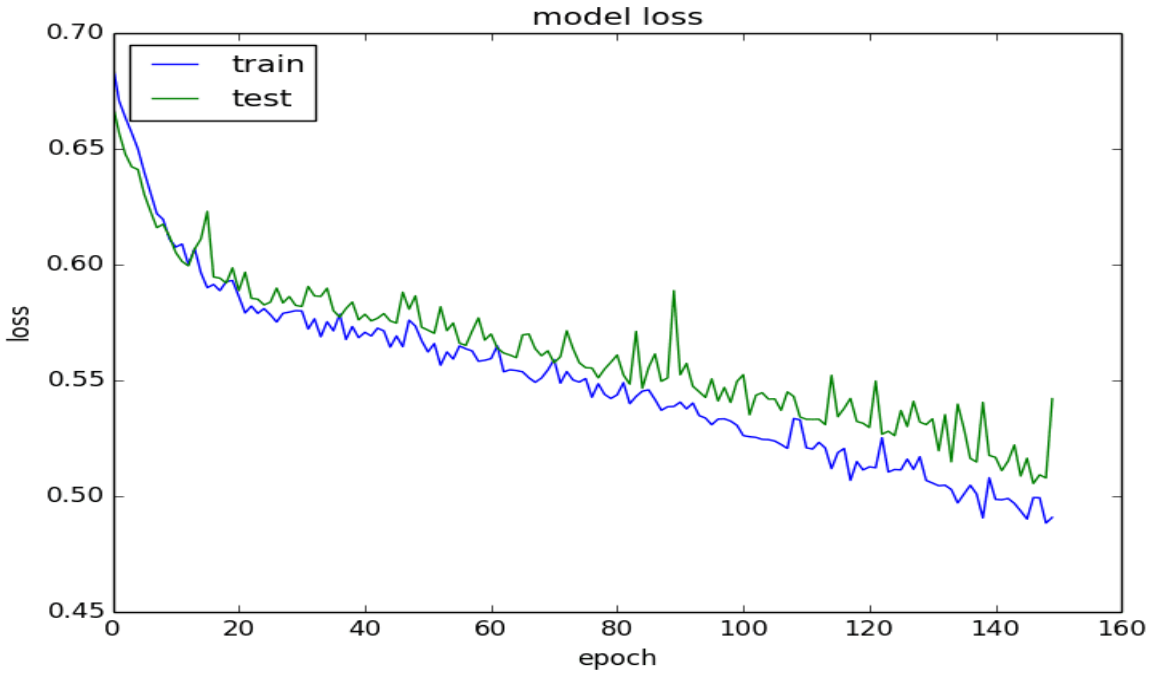


Figure 4.11: Plot of model loss on train and validation datasets

4.5 Summary : Discussion of Challenges and Solutions

No project of this scale is without challenges. This section details the key technical hurdles encountered during development and the engineered solutions that were implemented to overcome them.

- **Challenge 1: Slow Historical Data Retrieval**
    - **Problem:** During system testing with a simulated large dataset, it was observed that the initial load time for the client dashboard was unacceptably slow. The root cause was that the backend PHP script was attempting to fetch and process the entire historical dataset for a client every time the dashboard was loaded.
    - **Solution:** A two-part solution was engineered. Firstly, **database indexing** was applied to the system_id and timestamp columns of the sensor_logs table.
    - This dramatically accelerated the speed at which the MySQL server could query for specific data ranges. Secondly, the frontend-backend interaction was optimized.

- **Challenge 2: Unreliable Network Connectivity in the Field**
    - **Problem:** A key requirement was data reliability. However, the GSM cellular network can be intermittent in some remote or rural areas where Kajatronics installs systems (Akpan, 2024).. A simple fire and forget data transmission from the IoT unit could lead to significant data loss during periods of poor network coverage.
    - **Solution:** A store and forward mechanism was programmed into the Arduino firmware. The microcontroller now attempts to send its data to the server. If it does not receive an HTTP 200 OK success code in response, it saves the failed data packet (the JSON string) to its non-volatile EEPROM memory. On the next successful connection, it first transmits the old, stored data packet before sending the current one, ensuring that no data points are lost.
- **Challenge 3: Ensuring User Friendly Data Visualization**
    - **Problem:** Early feedback from User Acceptance Testing (UAT) indicated that while the data was present, some of the charts and metrics were not immediately intuitive to non-technical clients (Witczak, 2024).
    - **Solution:** The dashboard UI/UX was iteratively refined based on this feedback. Chart labels were simplified, tooltips were added to provide more context when a user hovers over a data point, and a color-coding system was implemented (e.g., green for normal status, amber for warnings) to make the overall system health instantly understandable.

Key Findings

The key findings, based on the concrete results and technical discussion, are:

1. **Successful Integration of Technical and Corporate Objectives:** The project successfully delivered a dual-purpose platform, bridging the gap between technical reliability (monitoring/predictive maintenance) and strategic organizational visibility (the corporate website).

2. **Validation of Predictive Alerts:** The rule-based predictive maintenance engine successfully generates actionable, human-readable alerts (e.g., High Temperature, Efficiency Drop) and provides a **Next Scheduled Maintenance** date, demonstrating the shift from reactive to proactive maintenance.

3. **Enhanced Data Reliability in Remote Areas:** The implementation of a **Store and Forward mechanism** in the Arduino firmware for the IoT unit (Challenge 2 solution) is critical. It guarantees that no data is lost during periods of poor GSM network connectivity by saving and re-transmitting failed data packets from non-volatile EEPROM memory.

CHAPTER 5:

5.0 CONCLUSION AND RECOMMENDATIONS

5.1 CONCLUSION

This project successfully delivered the LivingStone Connect platform, a centralized web-based solution that addressed Kajatronics Private Limited's dual challenges of insufficient real-time system monitoring and an underdeveloped digital presence. The implementation was guided by the Mechatronics VDI 2206 methodology and successfully achieved all five primary objectives.

Key Achievements:

1. **Centralized Platform and Digital Identity:** The comprehensive, multi-page corporate website, developed using HTML, CSS, JavaScript, PHP, and MySQL, successfully created a professional digital identity for Kajatronics. It functions as a strategic corporate hub, showcasing projects and services, thereby bridging the Weak Digital Identity gap and supporting client acquisition.

2. **Real-Time Monitoring and Dashboards:** The secure, role-based Client Dashboard was implemented to display essential real-time solar performance metrics (e.g., Current Power Output and Today's Energy Production), giving clients unprecedented transparency and improving customer satisfaction. The initial challenge of slow retrieval of historical data was mitigated through database indexing and API optimization (lazy loading).

3. **Predictive Maintenance Integration:** The core innovation, a Python-based predictive maintenance engine utilizing a rule-based classification algorithm, was successfully deployed as a cron job on the server. This system automates the analysis of sensor data, enabling the generation of actionable, human-readable alerts (e.g., "online status," "current power output") and displaying a next Scheduled Maintenance date, thereby successfully transitioning the company's approach from costly reactive maintenance to proactive, condition-based maintenance.

4. **Hardware Autonomy and Reliability:** The bespoke IoT Data Acquisition Unit, built on the Arduino platform with GSM connectivity, was proven feasible. Crucially, the implementation of a Store and Forward mechanism in the firmware ensures that no data is lost during periods of

poor cellular network connectivity, validating the platform's reliability for remote deployment in the Zimbabwean context.

In conclusion, the LivingStone Connect platform is a robust, cost-effective, and strategically important organizational hub. By integrating technical reliability with corporate visibility, the system is set to improve Kajatronics' operational efficiency, reduce system downtime, minimize long-term maintenance costs, and significantly strengthen its overall market competitiveness in the sustainable energy and automation sectors.

5.2 RECOMMENDATIONS

In the spirit of continuous improvement and building upon the successful foundation of the LivingStone Connect platform, the following recommendations are proposed for future development and strategic expansion:

1. Overhaul of Real-Time Data Ingestion Pipeline

The current PHP-based backend, despite initial optimizations, remains a bottleneck for high-volume data ingestion, leading to slow to no response when handling rapid sensor data transmission. To scale effectively, the system requires a specialized architecture:(Pereira, 2024; Chen et al., 2025).

- **Dedicated Data Endpoint:** Offload raw sensor data ingestion from the main PHP application server. Implement a lightweight message queue protocol, such as **MQTT**, and a dedicated, non-blocking Python or Node.js API to receive and quickly log the data.
- **Time-Series Database:** Migrate the core sensor data logs to a time-series optimized database (e.g., InfluxDB) or a specialized column store within MySQL, drastically improving write/read speeds for high-frequency time-stamped entries.

2. Enhance Hardware Resilience and Connectivity

- **Explore LPWAN Technologies:** Given the documented reliance on GSM network coverage, the next iteration of the IoT Data Acquisition Unit should investigate integrating or switching to Low-Power Wide-Area Network (LPWAN) technologies like LoRaWAN for deployments in ultra-remote, low-signal areas, which is better suited for small, periodic data packets. (Zhao, 2024)

4. Develop the Full Administration Portal and Mobile Interface

- **Implement Admin Diagnostics:** Fully develop the administrative dashboard to include fleet-wide system status, geospatial mapping of client installations, and deep diagnostic tools that allow technicians to remotely clear alerts, schedule maintenance, and update client system configurations, reducing the need for routine on site visits.
- **Dedicated Mobile Application:** Develop a native or progressive web app (PWA) version of the Client Dashboard and Admin Portal to ensure a seamless and optimized User Experience (UX) for on the go monitoring and management by both clients and Kajatronics technicians.

References

- Akpan, J. (2024) 'Sustainable energy in Zimbabwe – status, challenges and opportunities', *Regional Energy Journal*, 12(2), pp. 87–99.
- Awad, A.H. (2024) 'Low-cost IoT-based sensors dashboard for monitoring', *Engineering Reports*, 6(1), e1275.
- Barbaro, A. (2024) 'Data acquisition, processing, and aggregation in a low-cost monitoring system', *Applied Sciences*, 14(2), pp. 233–245.
- Chen, L., Wang, Y., Patel, S. and Gomez, R. (2025) 'Optimized predictive maintenance for streaming data in industrial IoT', *Scientific Reports*, 15(1), pp. 122–138.
- Chikowore, T. and Dube, M. (2021) 'Adoption of IoT for smart agriculture in Zimbabwe', *African Journal of ICT*, 17(3), pp. 77–89.
- Hassan, A.K. (2024) 'Low-cost IoT air quality monitoring station using cloud integration', *Sensors and Applications*, 13(1), pp. 100–114.
- Lee, H. and Kim, J. (2022) 'Condition monitoring of inverter power devices based on electromagnetic acoustic emissions', *Journal of Power Electronics*, 22(5), pp. 453–467.

- Ngwenya, P. (2023) ‘Secure remote monitoring as a service for solar power plants’, *Journal of Energy Systems*, 19(2), pp. 210–225.
- Parmar, A. (2024) ‘End-to-end low-cost IoT system for dense deployments’, *Frontiers in IoT*, 2(1), pp. 14–27.
- Pech, M., Vrchota, J. and Bednář, J. (2021) ‘Predictive maintenance and intelligent sensors in smart factories: review’, *Sensors*, 21(4), pp. 1470–1486.
- Pereira, F. (2024) ‘Machine learning for monitoring and classification in solar farms’, *Renewable Energy Analytics*, 5(1), pp. 23–39.
- Syamsuddin, A. (2024) ‘Anomaly detection-based predictive maintenance for photovoltaic systems’, *Energy Reports*, 10, pp. 55–67.
- Wiley Editorial (2024) ‘Trend-based predictive maintenance and fault detection for PV systems’, *Wiley Solar Research*, 9(2), pp. 199–212.
- Witczak, D. (2024) ‘Review of monitoring and control systems based on IoT’, *Applied Sciences*, 14(6), pp. 546–559.
- Zhao, Z. (2024) ‘Development and application of IoT monitoring systems’, *International Journal of Smart Systems*, 8(4), pp. 221–235.
- Zero, E. (2024) ‘Predictive maintenance in IoT-monitored systems’, *Internet of Things Journal*, 9(3), pp. 45–60
- Akpan, J. (2024). ‘Sustainable energy in Zimbabwe – status, challenges and opportunities’. *Regional Energy Journal*, 12(2), pp. 87–99.
- Arduino. (2023). *Arduino Uno R3 Official Documentation*. Available at: https://docs.arduino.cc/hardware/uno-rev3
- Awad, A.H. (2024). ‘Low-cost IoT-based sensors dashboard for monitoring’. *Engineering Reports*, 6(1), e1275.
- Barbaro, A. (2024). ‘Data acquisition, processing, and aggregation in a low-cost monitoring system’. *Applied Sciences*, 14(2), pp. 233–245.

- Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley Professional.
- Chen, L., Wang, Y., Patel, S., & Gomez, R. (2025). ‘Optimized predictive maintenance for streaming data in industrial IoT’. *Scientific Reports*, 15(1), pp. 122–138.
- Chikowore, T., and Dube, M. (2021). ‘Adoption of IoT for smart agriculture in Zimbabwe’. *African Journal of ICT*, 17(3), pp. 77–89.
- Freeman, E., & Robson, E. (2014). *Head First Design Patterns: A Brain-Friendly Guide*. O'Reilly Media.
- Hassan, A.K. (2024). ‘Low-cost IoT air quality monitoring station using cloud integration’. *Sensors and Applications*, 13(1), pp. 100–114.
- Haverbeke, M. (2018). *Eloquent JavaScript* (3rd ed.). No Starch Press.
- Lee, H., & Kim, J. (2022). ‘Condition monitoring of inverter power devices based on electromagnetic acoustic emissions’. *Journal of Power Electronics*, 22(5), pp. 453–467.
- McKinney, W. (2017). *Python for Data Analysis* (2nd ed.). O'Reilly Media.
- Ngwenya, P. (2023). ‘Secure remote monitoring as a service for solar power plants’. *Journal of Energy Systems*, 19(2), pp. 210–225.
- Nixon, R. (2021). *Learning PHP, MySQL & JavaScript: A Step-by-Step Guide to Creating Dynamic Websites* (6th ed.). O'Reilly Media.
- Parmar, A. (2024). ‘End-to-end low-cost IoT system for dense deployments’. *Frontiers in IoT*, 2(1), pp. 14–27.
- Pech, M., Vrchota, J., & Bednář, J. (2021). ‘Predictive maintenance and intelligent sensors in smart factories: review’. *Sensors*, 21(4), pp. 1470–1486.
- Pereira, F. (2024). ‘Machine learning for monitoring and classification in solar farms’. *Renewable Energy Analytics*, 5(1), pp. 23–39.
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- SIMCom Wireless Solutions. (2020). *SIM800L Hardware Design Manual*. Version 1.10.
- Syamsuddin, A. (2024). ‘Anomaly detection-based predictive maintenance for photovoltaic systems’. *Energy Reports*, 10, pp. 55–67.

- The Bootstrap Team. (2023). *Bootstrap 5 Documentation*. Available at: https://getbootstrap.com/docs/5.3/
- Witczak, D. (2024). ‘Review of monitoring and control systems based on IoT’. *Applied Sciences*, 14(6), pp. 546–559.
- Zhao, Z. (2024). ‘Development and application of IoT monitoring systems’. *International Journal of Smart Systems*, 8(4), pp. 221–235.

APPENDICES:

1:THE HTML CODE FOR MULTIPAGE WEBSITE

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="utf-8">
    <title>www.kajatronics.com</title>
    <meta content="width=device-width, initial-scale=1.0" name="viewport">
    <meta content="" name="keywords">
    <meta content="" name="description">
    <link rel="stylesheet" href="style.css">

    <!-- Favicon -->
    <link href="img/favicon.ico" rel="icon">

    <!-- Google Web Fonts -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Open+Sans:wght@400;500&family=Roboto:wgh
t@500;700;900&display=swap" rel="stylesheet">

    <!-- Icon Font Stylesheet -->
    <link href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/5.10.0/css/all.min.css" rel="stylesheet">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.4.1/font/bootstrap-
icons.css" rel="stylesheet">
```

```
    <!-- Libraries Stylesheet -->
    <link href="lib/animate/animate.min.css" rel="stylesheet">
    <link href="lib/owlcarousel/assets/owl.carousel.min.css" rel="stylesheet">
    <link href="lib/lightbox/css/lightbox.min.css" rel="stylesheet">

    <!-- Customized Bootstrap Stylesheet -->
    <link href="css/bootstrap.min.css" rel="stylesheet">

    <!-- Template Stylesheet -->
    <link href="css/style.css" rel="stylesheet">
</head>

<body>
    <!-- Spinner Start -->
    <div id="spinner" class="show bg-white position-fixed translate-middle w-100 vh-100
top-50 start-50 d-flex align-items-center justify-content-center">
        <div class="spinner-border text-primary" style="width: 3rem; height: 3rem;"
role="status">
            <span class="sr-only">Loading...</span>
        </div>
    </div>
    <!-- Spinner End -->


    <!-- Topbar Start -->
    <div class="container-fluid bg-dark p-0">
        <div class="row gx-0 d-none d-lg-flex">
            <div class="col-lg-7 px-5 text-start">
                <div class="h-100 d-inline-flex align-items-center me-4">
                    <small class="fa fa-map-marker-alt text-primary me-2"></small>
                    <small>95 Mutare Road, Msasa, Zimbabwe</small>
                </div>
                <div class="h-100 d-inline-flex align-items-center">
                    <small class="far fa-clock text-primary me-2"></small>
                    <small>Mon - Fri : 08.00 AM - 05.00 PM</small>
                </div>
            </div>
            <div class="col-lg-5 px-5 text-end">
                <div class="h-100 d-inline-flex align-items-center me-4">
                    <small class="fa fa-phone-alt text-primary me-2"></small>
                    <small>+263 783 154 354 <br></small>
                </div>
                <div class="h-100 d-inline-flex align-items-center mx-n2">
```

```
                    <a class="btn btn-square btn-link rounded-0 border-0 border-end
border-secondary" href="https://www.facebook.com/share/196nqoLQm5/?mibextid=wwXlfr"><i
class="fab fa-facebook-f"></i></a>
                    <a class="btn btn-square btn-link rounded-0 border-0 border-end
border-secondary" href=""><i class="fab fa-twitter"></i></a>
                    <a class="btn btn-square btn-link rounded-0 border-0 border-end
border-secondary" href=""><i class="fab fa-linkedin-in"></i></a>
                    <a class="btn btn-square btn-link rounded-0"
href="https://www.instagram.com/kajatronics?igsh=MXY2eXhneXJ5M2lsMw%3D&utm_source=qr"><
i class="fab fa-instagram"></i></a>
                </div>
            </div>
        </div>
    </div>
    <!-- Topbar End -->


    <!-- Navbar Start -->
    <nav class="navbar navbar-expand-lg bg-white navbar-light sticky-top p-0">
        <a href="index.html" class="navbar-brand d-flex align-items-center border-end
px-4 px-lg-5">
            <h2 class="m-0 text-primary">Kajatronics</h2>
        </a>
        <button type="button" class="navbar-toggler me-4" data-bs-toggle="collapse"
data-bs-target="#navbarCollapse">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="navbarCollapse">
            <div class="navbar-nav ms-auto p-4 p-lg-0">
                <a href="index.html" class="nav-item nav-link active">Home</a>
                <a href="about.html" class="nav-item nav-link">About</a>
                <a href="service.html" class="nav-item nav-link">Services</a>
                <a href="project.html" class="nav-item nav-link">Projects</a>
                <div class="nav-item dropdown">
                    <a href="#" class="nav-link dropdown-toggle" data-bs-
toggle="dropdown">Pages</a>
                    <div class="dropdown-menu bg-light m-0">
                        <a href="feature.html" class="dropdown-item">Feature</a>
                        <a href="quote.html" class="dropdown-item">Free Quote</a>
                        <a href="team.html" class="dropdown-item">Our Team</a>
                        <a href="testimonial.html" class="dropdown-
item">Testimonial</a>
                        <a href="404.html" class="dropdown-item">404 Page</a>
                    </div>
```

THE CSS CODE FOR THE MUTLIPAGE WEBSITE

```
/**********  CSS **********/
:root {
    --primary: #5FBF05;
    --light: #F6F7F8;
    --dark: #1A2A36;
}

.fw-medium {
    font-weight: 500 !important;
}

.fw-bold {
    font-weight: 700 !important;
}

.fw-black {
    font-weight: 900 !important;
}

.back-to-top {
    position: fixed;
    display: none;
    right: 45px;
    bottom: 45px;
    z-index: 99;
}


/*** Spinner ***/
#spinner {
    opacity: 0;
    visibility: hidden;
    transition: opacity .5s ease-out, visibility 0s linear .5s;
    z-index: 99999;
}

#spinner.show {
    transition: opacity .5s ease-out, visibility 0s linear 0s;
    visibility: visible;
    opacity: 1;
}
```

```
/*** Button ***/
.btn {
    font-weight: 500;
    transition: .5s;
}

.btn.btn-primary,
.btn.btn-outline-primary:hover {
    color: #FFFFFF;
}

.btn-square {
    width: 38px;
    height: 38px;
}

.btn-sm-square {
    width: 32px;
    height: 32px;
}

.btn-lg-square {
    width: 48px;
    height: 48px;
}

.btn-square,
.btn-sm-square,
.btn-lg-square {
    padding: 0;
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: normal;
}

/*** Navbar ***/
.navbar.sticky-top {
    top: -100px;
    transition: .5s;
}

.navbar .navbar-brand,
```

```
.navbar a.btn {
    height: 75px;
}

.navbar .navbar-nav .nav-link {
    margin-right: 30px;
    padding: 25px 0;
    color: var(--dark);
    font-size: 15px;
    font-weight: 500;
    text-transform: uppercase;
    outline: none;
}

#start{
    color:#ff7220;
}

.navbar .navbar-nav .nav-link:hover,
.navbar .navbar-nav .nav-link.active {
    color: var(--primary);
}

.navbar .dropdown-toggle::after {
    border: none;
    content: "\f107";
    font-family: "Font Awesome 5 Free";
    font-weight: 900;
    vertical-align: middle;
    margin-left: 8px;
}

@media (max-width: 991.98px) {
    .navbar .navbar-nav .nav-link  {
        margin-right: 0;
        padding: 10px 0;
    }

    .navbar .navbar-nav {
        border-top: 1px solid #EEEEEE;
```

THE CODE FOR CONTACT FORM

```
<!--form start-->
                        <div class="form">
                           <h1></h1>
                           <form
action="https://formsubmit.co/tinodaisheguveya2000@gmail.com" method="POST">

                              <div class="row g-3">
                                   <div class="col-md-6">
                                   <div class="form-floating">
                                   <input type="text" class="form-
control"  name="name" required placeholder="Enter your name">
                                   <label for="name">Your Name</label>
                                  </div>
                              </div>

                              <div class="col-md-6">
                                  <div class="form-floating">
                                  <input type="email" class="form-control"
name="email" required placeholder=" Enter your email">
                                   <label for="email">your email</label>
                                  </div>
                              </div>


                              <div class="col-12">
                                  <div class="form-floating">
                                      <input type="text"  class="form-control"
name="subject" required placeholder="Enter your subject">
                                      <label for="subject">Subject</label>
                                  </div>
                              </div>

                               <div class="col-12">
                                  <div class="form-floating">
                                      <input type="text"  class="form-control"
name="phone number" required placeholder="Enter your (code) phone number ">
                                      <label for="phone number">Phone Number</label>
                                  </div>
                               </div>

                               <div class="col-12">
                                  <div class="form-floating">
```

```
                                        <textarea  class="form-control"  name="message"
required placeholder="Enter your message"></textarea>
                                        <label for="message">message</label>
                                    </div>
                                 </div>
                                </div>

                                <input type="hidden" name="_captcha" value="false">

                                <input type="hidden" name="_next"
value="https://kajatronics.co/thanks.html">
                                <input type="hidden" name="_template"
value="basic"><br>
                                <button type="submit" class="btn btn-primary rounded-
pill py-3 px-5" >submit</button>
                            </form>
                        </div>
                        <!--form end-->
```

USER LOGIN DASHBOARD

```
      <!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Kajatronics Client Dashboard</title>
    <link rel="stylesheet" href="styles/style2.css">
    <link rel="stylesheet" href="style.css">
</head>
<body>

     <!-- CLIENT LOGIN SECTION -->
    <section class="login-form" id="client-login">
        <h2>Client Access</h2>
        <form onsubmit="return validateClientAccess()">
            <label for="client-name">Full Name:</label>
            <input type="text" id="client-name" required>

            <label for="client-id">National ID Number:</label>
            <input type="text" id="client-id" required>
```

```
            <label for="client-system">System Size (e.g., 3kVA, 8kVA):</label>
            <input type="text" id="client-system" required>

            <button type="submit">Access Dashboard</button>
        </form>
    </section>


    <button onclick="showClientLoginContents()">Check System Health</button>



<script src="script.js"></script>
</body>
</html>
```

THE CODE FOR PREDICTIVE MAINTENANCE

The python predictive engine (ML) program:

```
# predictive_engine.py

# Description: This script connects to the production database, analyzes recent sensor data

# for all active systems, and creates predictive maintenance alerts based on

# a set of pre-defined rules. It is designed to be run automatically as a cron job.



import mysql.connector

import os

import logging

from datetime import datetime, timedelta

from dotenv import load_dotenv



# --- Configuration ---

# Load environment variables from a .env file for security
```

```
load_dotenv()


# Database credentials
DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_USER = os.getenv('DB_USER')
DB_PASSWORD = os.getenv('DB_PASSWORD')
DB_NAME = os.getenv('DB_NAME')


# Logging configuration
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')


# --- Rule Thresholds ---
# These constants define the triggers for our predictive alerts.
# They are placed at the top for easy tuning and maintenance.
HIGH_BATT_TEMP_THRESHOLD = 45.0  # Celsius
EFFICIENCY_DROP_PERCENTAGE = 0.95 # Corresponds to a 5% drop
EFFICIENCY_BASELINE_DAYS = 90    # Days for the historical average
EFFICIENCY_RECENT_DAYS = 30      # Days for the recent average


# --- Database Interaction Functions ---


def create_db_connection():
    """Creates and returns a connection to the MySQL database."""
```

```
    try:
        conn = mysql.connector.connect(
            host=DB_HOST,
            user=DB_USER,
            password=DB_PASSWORD,
            database=DB_NAME
        )
        if conn.is_connected():
            logging.info("Successfully connected to the database.")
            return conn
    except mysql.connector.Error as e:
        logging.error(f"Error connecting to MySQL Database: {e}")
        return None


def get_active_systems(conn):
    """Fetches a list of all active system IDs from the database."""
    cursor = conn.cursor()
    try:
        cursor.execute("SELECT id FROM systems WHERE status = 'active'")
        systems = [item[0] for item in cursor.fetchall()]
        logging.info(f"Found {len(systems)} active systems to analyze.")
        return systems
    except mysql.connector.Error as e:
```

```
        logging.error(f"Error fetching active systems: {e}")
        return []
    finally:
        cursor.close()


def get_sensor_data(conn, system_id, days):
    """Fetches sensor data for a specific system over a given number of days."""
    cursor = conn.cursor(dictionary=True)
    try:
        start_date = datetime.now() - timedelta(days=days)
        query = """
            SELECT timestamp, inverter_v, inverter_a, panel_a, batt_t
            FROM sensor_logs
            WHERE system_id = %s AND timestamp >= %s
            ORDER BY timestamp ASC
        """
        cursor.execute(query, (system_id, start_date))
        return cursor.fetchall()
    except mysql.connector.Error as e:
        logging.error(f"Error fetching sensor data for system {system_id}: {e}")
        return []
    finally:
        cursor.close()
```

```
def create_alert(conn, system_id, alert_type, message):
    """Inserts a new alert into the database if a similar one doesn't already exist."""
    cursor = conn.cursor()
    try:
        # First, check for an existing, unacknowledged alert of the same type to avoid duplicates
        check_query = "SELECT id FROM alerts WHERE system_id = %s AND alert_type = %s
AND status = 'new'"
        cursor.execute(check_query, (system_id, alert_type))
        if cursor.fetchone():
            logging.info(f"An active alert of type '{alert_type}' already exists for system {system_id}.
Skipping.")
            return

        # If no active alert exists, insert a new one
        insert_query = """
            INSERT INTO alerts (system_id, alert_type, message, status, timestamp)
            VALUES (%s, %s, %s, 'new', %s)
        """
        cursor.execute(insert_query, (system_id, alert_type, message, datetime.now()))
        conn.commit()
        logging.warning(f"ALERT CREATED for system {system_id}: {message}")
    except mysql.connector.Error as e:
```

```
        logging.error(f"Error creating alert for system {system_id}: {e}")
    finally:
        cursor.close()


# --- Predictive Rule Functions ---


def check_high_battery_temperature(conn, system_id):
    """Rule 1: Checks for dangerously high battery temperatures in the last 24 hours."""
    logging.info(f"Running rule [High Temp] for system {system_id}...")
    data = get_sensor_data(conn, system_id, days=1)
    if not data:
        return

    for reading in data:
        if reading['batt_t'] and reading['batt_t'] > HIGH_BATT_TEMP_THRESHOLD:
            message = (f"High temperature alert! Battery reached {reading['batt_t']:.1f}°C. "
                       "Please check system ventilation and ambient temperature.")
            create_alert(conn, system_id, "High Temperature", message)
            break # One alert is enough for this period


def check_solar_panel_efficiency_drop(conn, system_id):
    """Rule 2: Checks for a significant drop in solar panel power generation."""
    logging.info(f"Running rule [Efficiency Drop] for system {system_id}...")
```

```
    total_days = EFFICIENCY_BASELINE_DAYS + EFFICIENCY_RECENT_DAYS
    data = get_sensor_data(conn, system_id, days=total_days)

    if len(data) < 50: # Not enough data to make a meaningful comparison
        logging.info(f"Skipping efficiency check for system {system_id} due to insufficient data.")
        return

    baseline_end_date = datetime.now() - timedelta(days=EFFICIENCY_RECENT_DAYS)

    baseline_power = []
    recent_power = []

    for reading in data:
        # Calculate power. We use panel current and inverter voltage as a proxy for panel output power.
        # This is an effective way to track relative performance drops.
        if reading['inverter_v'] and reading['panel_a']:
            power = reading['inverter_v'] * reading['panel_a']
            if reading['timestamp'] < baseline_end_date:
                baseline_power.append(power)
            else:
                recent_power.append(power)
```

```
    if not baseline_power or not recent_power:
        logging.info(f"Skipping efficiency check for system {system_id}: not enough data in both periods.")
        return

    avg_baseline_power = sum(baseline_power) / len(baseline_power)
    avg_recent_power = sum(recent_power) / len(recent_power)

    if avg_baseline_power > 0 and avg_recent_power < (avg_baseline_power * EFFICIENCY_DROP_PERCENTAGE):
        percentage_drop = (1 - (avg_recent_power / avg_baseline_power)) * 100
        message = (f"Performance drop detected! Recent power generation is approx. {percentage_drop:.0f}% lower "
                   "than the historical baseline. Solar panels may require cleaning or inspection.")
        create_alert(conn, system_id, "Efficiency Drop", message)

# --- Main Execution Logic ---

def main():
    """Main function to orchestrate the entire analysis process."""
    logging.info("--- Starting Predictive Maintenance Engine Run ---")

    conn = create_db_connection()
    if not conn:
```

```
        logging.critical("Could not establish database connection. Aborting run.")
        return

    try:
        active_systems = get_active_systems(conn)
        for system_id in active_systems:
            logging.info(f"--- Analyzing System ID: {system_id} ---")
            check_high_battery_temperature(conn, system_id)
            check_solar_panel_efficiency_drop(conn, system_id)
            # Add calls to new rule functions here in the future
            # e.g., check_inverter_overload(conn, system_id)
    finally:
        if conn.is_connected():
            conn.close()
            logging.info("Database connection closed.")

    logging.info("--- Predictive Maintenance Engine Run Finished ---")

if __name__ == "__main__":
    main()
```